Ironwood Guide

The Zcash Orchard protocol and its Ironwood pool

m@rek.onl

Abstract

This volume constructs the Zcash Orchard shielded-payment protocol as carried by its current value pool, Ironwood. The presentation follows one unit of value through its lifecycle: a recipient prepares keys and an address, a note is minted and delivered, it rests in the commitment tree, it is spent — retiring its nullifier, balancing its value commitment, authorised by its re-randomised signature — change returns, and value crosses the boundary between the sealed Orchard pool and Ironwood. Each piece of machinery is constructed at the lifecycle stage that first demands it and assembled into the Action statement every shielded transfer proves; the volume then descends to the circuit beneath the statement, pays the deferred security debts in end-to-end reductions, and closes with the proof-system lineage and the quantum horizon. It assumes the *Math Guide*, the *Crypto Guide*, and the *Halo 2 Guide*, and is non-normative throughout: the protocol specification and the ZIPs remain authoritative.

Contents

1 Introduction: the life of a shielded zatoshi

Halo 2 is a general-purpose proving system, and it has no notion of money. It can attest that a prover knows a witness satisfying a fixed arithmetic relation, but nothing in it says what a coin is, who owns one, or when one has been spent twice. The Orchard application turns that general instrument into a shielded-payment protocol by one act of specification: it fixes a single relation — the *Action statement* — that every shielded transfer must prove, and lets the consensus rules of a public chain enforce the rest. This volume constructs that relation and everything around it, deriving each cryptographic object as the answer to one sub-problem, tracing a complete payment from Alice to Bob end to end, and only then stating the Action relation formally (§8). It assumes three earlier Arboretum volumes: the *Math Guide* for the underlying mathematics, the *Crypto Guide* for hash functions, commitments, signatures, key agreement, zero knowledge, and the rest of the primitive toolbox, and the *Halo 2 Guide* for the proof system itself. Here those tools are assembled into the Orchard protocol as carried by its current pool, Ironwood — and the protagonist of the assembly is one *zatoshi*, the atomic unit of ZEC, Zcash's native currency, followed from the moment it turns shielded.

1.1 Two pools, one protocol

Zcash changes its consensus rules by *network upgrades*: scheduled rule changes, each activating at a block height this volume never needs numerically and carrying a name — NU5, NU6.3 — that serves only to order events. A *Zcash Improvement Proposal* (ZIP) is the document form in which protocol changes are specified; ZIP-229 fixes the vocabulary used throughout. The *Orchard protocol* is the shared cryptographic construction — the curves, Sinsemilla, the Action circuit, notes, commitments, nullifiers, keys, and note encryption, an inventory of names the roadmap (§1.3) assigns to the sections that construct them — and it survives the pool transition. The *Orchard pool* is the legacy value pool: it has run the Orchard protocol since NU5, and at NU6.3 it is sealed to spend-only. The *Ironwood pool* is the current pool, carrying the same protocol. One protocol, two carriers: when a statement holds for the shared cryptography we say "Orchard", and when it depends on which pool a note lives in we name the pool.

Value reaches a shielded pool through three doors, each public in amount. A *shielding* transaction spends transparent value into the pool; *shielded coinbase* lets the block reward — freshly issued coins — arrive shielded directly; and *migration* moves value from another shielded pool, which for this volume's two pools means out of sealed Orchard into Ironwood. All three doors are formalised in §9; their publicity is by design, for what a pool hides is who holds which note, never how much the pool as a whole contains.

1.2 The four requirements on a shielded payment

A transparent ledger — Bitcoin, or Zcash's transparent pool — records, for every unspent output, its owner and its denomination, and a payment publicly transfers an output between owners. A shielded payment must achieve the same effects — transfer of value, prevention of double-spending, conservation of total supply — while disclosing none of them. Four requirements must hold simultaneously:

(R1) represent a unit of value so that its owner, its value, and its very existence stay hidden, yet the owner can later prove possession — solved by notes and note commitments (§4);

(R2) prove possession of a real, unspent note without identifying which note — solved by the commitment tree with a membership proof (§6);

(R3) prevent double-spending without a public list of which notes an owner has spent — solved by nullifiers (§7);

(R4) prove that no one created value without revealing any amount — solved by value commitments and the binding signature (§7).

Each stage of the journey ahead opens by naming the requirement it discharges, and the loop is closed at the end: §12 proves that the assembled protocol meets all four.

1.3 Method and roadmap: the journey

A protocol this dense can be presented from its symbols or from the problems it solves; we take the second way, and give it a narrative spine. The volume follows one unit of value through the system — a recipient prepares to be paid, a note is minted and delivered, it rests in the commitment tree, it is spent, change returns, and at the end of its shielded life value exits back across the pool boundary, in public amount — with each piece of protocol machinery constructed at the lifecycle stage that first demands it.

After the curve-level preliminaries and the Sinsemilla hash are packed once (§2), each object arrives as the answer to one sub-problem. The recipient's keys and addresses come first (§3). Notes and note commitments represent value (§4); note encryption and trial decryption deliver a note privately and let its owner detect it (§5); the commitment tree and its membership path prove ownership of a real unspent note without revealing which (§6); nullifiers prevent double-spending, value commitments and the binding signature conserve value without disclosure, and the RedPallas signature scheme authorises the spend (§7). The unit these pieces assemble into is the *Action* — one Action spends one old note and creates one new one — and §8 walks one payment end to end, shows how a node verifies it without seeing any note, and states the relation the proof carries. The two pools and their doors follow (§9), then the seal and the turnstile that empty the legacy pool (§10), the circuit beneath the statement (§11), and the end-to-end security reductions that pay the volume's deferred debts (§12). A closing pair of sections looks backward and forward: the lineage Sprout → Sapling → Orchard — three generations of shielded proof, and within Orchard three circuit versions ending at Ironwood (§13) — and the quantum horizon (§14). Figure 1 draws the map.

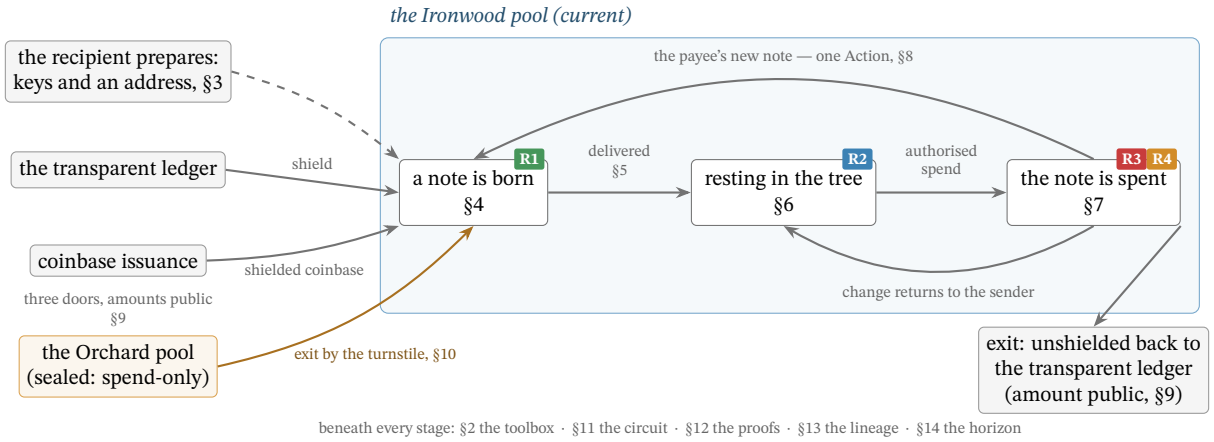

Figure 1: The volume's map: one zatoshi's lifecycle across the two pools. Value enters the Ironwood pool through three doors, each public in amount (§9); a recipient who has prepared keys and an address (§3) is paid with a freshly minted note (R1, §4), which is delivered in band (§5) and rests as a leaf of the commitment tree (R2, §6). Its spend (R3 and R4, §7) is one half of an Action (§8) whose products — the payee's new note and the sender's change — re-enter the cycle. The sealed Orchard pool moves value in one direction only, out through the turnstile (§10). At the end of its shielded life, value exits as it entered: unshielded back to the transparent ledger, in public amount (§9). Each stage is tagged with the section that constructs its machinery and the requirement of §1.2 it discharges.

2 Provisions: hashing on Pallas and Sinsemilla

Every stage of the journey ahead draws on the same two workhorses: a small kit of curve-level encodings and hash-to-curve maps, and the Sinsemilla hash built from them. Rather than interrupt the narrative to construct these where each stage first needs them, we pack them here, once — the one deliberate non-narrative cost of the volume, paid up front and kept tight. All curve-level primitives of this section operate on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$: the prime-order group of points of the Pallas curve, one half of the Pasta cycle of curves and fields constructed in the *Math Guide*, §"Elliptic curves".

2.1 Points to field elements: Extract, the star encoding, and LE_n

The coordinate map Extract**.** The map $\mathsf{Extract} : \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ sends a curve point to its x-coordinate, $\mathsf{Extract}(x, y) := x$, with $\mathsf{Extract}(\mathcal{O}) := 0$ by convention. It appears wherever a later object must be a *field element* rather than a curve point — the Merkle tree (§6), the nullifier set (§7), and the proof's public inputs (§8) are all field elements. The map is not injective ($\pm(x, y)$ share an x), which is harmless here: the protocol only ever requires the x-coordinate as a binding digest, never to recover the point.

The point encoding $\star$. The starred form $P^\star \in \{0,1\}^{256}$ is the canonical bit-encoding of a point $P \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$: its x-coordinate (an element of $\mathbb{F}_{p_{\mathsf{Pallas}}}$, which fits in 255 bits since $p_{\mathsf{Pallas}} < 2^{255}$) packed into a 256-bit string together with one sign bit, placed in the otherwise-unused top bit, recording which of the two square roots $\pm y$ is meant. Unlike Extract, this encoding is injective — it determines P completely — and it is what is fed, as a bit-string, into a hash or commitment.

Integer encodings LE_n**.** The string $\mathrm{LE}_n(m) \in \{0,1\}^n$ is the n-bit little-endian encoding of an integer $m \in \{0, \ldots, 2^n - 1\}$: the bit string $b_0 b_1 \cdots b_{n-1}$ with $m = \sum_{i=0}^{n-1} b_i \, 2^i$ (a field element encodes via its integer representative). Three widths recur: LE_{32} indexes the Sinsemilla generator table (§2.3), LE_{10} tags the Merkle-tree layers (§6), and LE_{64}, LE_{255} encode note values and field elements (§4).

2.2 GroupHash, domain separation, and nothing-up-my-sleeve generators

The hash-to-curve GroupHash**.** The map $\mathsf{GroupHash}^{\mathbb{G}} : \{0,1\}^* \times \{0,1\}^* \to \mathbb{G}$ takes a pair (domain separator D, message M) and returns a point of the group $\mathbb{G}$. It is *deterministic* (the same input always yields the same point), *public* (anyone can evaluate it), and *efficiently computable*. It instantiates the IETF hash-to-curve standard: hash_to_field expands (D, M) — via expand_message_xmd over BLAKE2b-512, with D embedded in the domain-separation tag — into *two* base-field elements; each passes through the simplified SWU map onto a curve isogenous to $\mathbb{G}$; and the sum of the two resulting points is carried through the isogeny to $\mathbb{G}$ (the *Crypto Guide*, §"Hashing to a curve point", develops this construction in full). Everything that follows uses only that GroupHash is deterministic, public, and behaves like a random choice of point.

That last property is the purpose of the construction. Because every stage is public and deterministic, the public strings D, M pin down the output $\mathsf{GroupHash}^{\mathbb{G}}(D, M)$, so no party can have chosen it to satisfy a hidden relation such as $P_2 = [s]P_1$ for a secret s; such points are called *nothing-up-my-sleeve* generators. For the security arguments we *model* GroupHash as a random oracle into $\mathbb{G}$ (the *Crypto Guide*, §"The random oracle model"): its outputs are treated as independent uniform points, so the family it produces has no efficiently-findable non-trivial discrete-log relation $\sum_i [a_i]P_i = \mathcal{O}$ (discrete logarithms: the *Crypto Guide*, §"The discrete logarithm problem") — precisely the assumption the Sinsemilla binding and collision-resistance reductions invoke (§12). As with all random-oracle arguments this is a strong, standard, but unprovable modelling heuristic about the concrete hash, not a theorem.

Domain separators. A *domain separator* $D \in \{0,1\}^*$ is a fixed ASCII byte string naming the use site, fed to GroupHash so that logically distinct uses draw independent, unrelated generators (a collision found in one context cannot be transported to another). The strings are part of the protocol specification; the ones relevant to this volume are

`z.cash : Orchard`	fixed generators $G^{\mathsf{SpendAuth}}, G^{\mathsf{nf}}$ (§3, §7),
`z.cash : Orchard-gd`	the diversified base $\mathsf{g_d}$ (§3),
`z.cash : Orchard-NoteCommit`	note commitments (§4),
`z.cash : Orchard-CommitIvk`	the ivk commitment (§3),
`z.cash : Orchard-MerkleCRH`	the Merkle-tree node hash (§6),
`z.cash : Orchard-cv`	the value-commitment bases (§7),
`z.cash : SinsemillaQ`, `z.cash : SinsemillaS`	the Sinsemilla Q point and S table (§2.3).

2.3 Sinsemilla: hash, commitment, and short forms

Two objects the journey will build — the note commitment (§4) and the Merkle-tree node hash (§6) — require the same primitive: a commitment/hash that the spender must recompute *inside the zero-knowledge circuit*, the arithmetised form (the *Halo 2 Guide*, §"PLONKish arithmetisation") of the spend statement (§8). In-circuit cost is therefore the binding constraint, and it is what Sinsemilla (Bowe and Hopwood) is designed to minimise. Its collision resistance (the *Crypto Guide*, §"Security notions: preimage, second-preimage, and collision resistance") reduces to discrete-log hardness on a prime-order curve, so it introduces no assumption beyond the DL hardness that Orchard's algebraic core already requires. And it builds directly on the provisions above: its generators are GroupHash outputs (§2.2), and its short form returns an Extracted coordinate (§2.1).

Construction 2.1 (Sinsemilla)**.** Fix a prime-order group $\mathbb{G}$ (Orchard: $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$) and a chunk width k ($k = 10$ in Orchard). From the domain separator D, derive the $2^k + 1$ Sinsemilla generators:

$$Q(D) := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash : SinsemillaQ}, D) \in \mathbb{G},$$
$$P[m] := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash : SinsemillaS}, \mathrm{LE}_{32}(m)) \in \mathbb{G}, \qquad 0 \le m < 2^k.$$

The point $Q(D)$ is the domain-dependent starting accumulator; the 2^k points $P[0], \ldots, P[2^k - 1]$ form a fixed table indexed by a k-bit chunk value, shared across all Sinsemilla domains. For a message M split into k-bit chunks $m_1, \ldots, m_\ell$ (so each $m_i \in \{0, \ldots, 2^k - 1\}$ indexes the table),

$$\mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell, \quad \mathsf{Acc}_0 := Q(D), \quad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \mathsf{Acc}_i,$$

with $\dot{+}$ *incomplete* short-Weierstrass addition (the *Math Guide*, §"Elliptic curves"). The output $\mathsf{Acc}_\ell \in \mathbb{G}$ is a curve point. The deployed hash zero-pads M on the right to a multiple of k bits before chunking (protocol specification § 5.4.1.9); every use in this volume fixes the input bit-length, and the padding is why Theorem 2.2 below is restricted to fixed-length inputs.

Theorem 2.2 (Sinsemilla collision resistance)**.** *For fixed-length inputs,* $\mathsf{Sinsemilla}_D$ *is collision-resistant assuming DL on* $\mathbb{G}$ *and modelling the generator derivation as a random oracle.*

The proof is deferred: Theorem 2.2 is proved in §12, by reducing a collision to a non-trivial discrete-log relation among the random-oracle-derived generators — the assumption named in §2.2. Every later security property of the volume follows the same pattern: stated where its object is constructed, proved once in §12.

Definition 2.3 (Sinsemilla commitment and short forms)**.** The *Sinsemilla commitment* adds a hiding term to the hash:

$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\,\|\,\texttt{-M}}(M) \dotplus [r]\,H_D \in \mathbb{G},$$

where $r \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment randomness (trapdoor; the Pasta cycle identifies $\mathbb{F}_{p_{\mathsf{Vesta}}}$ with the scalar field of Pallas), the hash runs in the suffixed domain $D\,\|\,\texttt{-M}$, and $H_D \in \mathbb{G}$ is a fixed blinding generator for the domain D, derived as the GroupHash output

$$H_D := \mathsf{GroupHash}^{\mathbb{G}}\big(D\,\|\,\texttt{-r},\ \varepsilon\big)$$

— the suffix `-r` appended to the domain string, with the empty message ε (a nothing-up-my-sleeve point, §2.2). The *short commitment* returns only its x-coordinate, a base-field element rather than a curve point:

$$\mathsf{ShortCommit}_r(D, M) := \mathsf{Extract}\big(\mathsf{SinsemillaCommit}_r(D, M)\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}}.$$

The unkeyed *short hash* omits the blinding term altogether:

$$\mathsf{ShortHash}_D(M) := \mathsf{Extract}\big(\mathsf{Sinsemilla}_D(M)\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$

the unblinded analogue of ShortCommit — no randomness argument, collision-resistant but not hiding. Collision resistance of the two short forms needs one case beyond Theorem 2.2: the theorem binds curve points, while Extract identifies a point with its negation (§2.1), so an x-coordinate collision may also be a pair of opposite points. Both cases are discharged together in §12.

Why is the commitment binding and hiding? Because H_D is a GroupHash output in its own subdomain ($D\,\|\,\texttt{-r}$), no party knows a discrete-log relation between it and the generators $Q(D\,\|\,\texttt{-M}), P[m]$ that the hash uses; this independence is exactly what keeps the commitment binding — a party knowing such a relation could trade the blinding term against the message and open one commitment to two distinct messages. Hiding needs no assumption at all: for uniform r the term $[r]H_D$ is uniform on $\mathbb{G}$ and masks the hash completely. This is the same division of roles the independent base H supports in a Pedersen commitment (the *Crypto Guide*, §"The Pedersen commitment").

The commitment is therefore *perfectly hiding* and *computationally binding* (the *Crypto Guide*, §"Hiding and binding") — the two properties §4 requires of NoteCommit. Indeed the note commitment $\mathsf{NoteCommit}_{\mathsf{rcm}}(\cdot)$ is exactly $\mathsf{SinsemillaCommit}_{\mathsf{rcm}}$ under the domain `z.cash : Orchard-NoteCommit`. The short forms appear wherever the consumer requires a field element instead of a point: ShortHash in the Merkle-tree node hash (§6), ShortCommit in the incoming viewing key ivk (§3). The latter use enters through a named instance that fixes the domain and the message encoding:

$$\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(x, y) := \mathsf{ShortCommit}_{\mathsf{r_{ivk}}}\big(\texttt{z.cash : Orchard-CommitIvk},\ \mathrm{LE}_{255}(x)\,\|\,\mathrm{LE}_{255}(y)\big),$$

taking two field elements and concatenating their 255-bit little-endian encodings into the 510-bit message. Its output lies in $\{1, \ldots, p_{\mathsf{Pallas}} - 1\}$: the zero output cannot occur for a valid key, so ivk is a usable scalar.

2.4 Why Sinsemilla: in-circuit cost

Each round of Construction 2.1 consumes a whole k-bit chunk via one table lookup of $P[m]$ and two incomplete additions, so a 510-bit message costs 51 rounds rather than 510 bit-operations. In a PLONKish circuit with lookups (the *Halo 2 Guide*, §"PLONKish arithmetisation" and §"The lookup argument") this is dramatically cheaper than the bit-by-bit Pedersen hash (the *Crypto Guide*, §"Com-

mitment schemes") of the previous shielded generation, Sapling — which is the reason the protocol changed hash between generations. The provisions are packed; the journey can now set out.

3 The recipient: keys and addresses

The journey begins with someone able to receive. Before a single zatoshi moves, the recipient must hold the material that later stages will lean on, so we build the Orchard machinery bottom-up, beginning with the keys: everything later — addresses, note commitments, nullifiers — derives from them. A practical requirement shapes the key hierarchy: different parties should be able to perform different operations. The owner can spend. An auditor should be able to *see* incoming and outgoing payments without being able to spend. A point-of-sale terminal should be able to *detect* incoming payments without seeing outgoing ones. Each of these is a distinct capability, and the key tree makes each capability a separate derived key, so the owner can hand out a lower-level key without surrendering the powers above it.

3.1 From sk to the spend-side secrets

Definition 3.1 (Spending key)**.** The root secret of the Orchard key hierarchy is a 32-byte *spending key* sk, chosen uniformly at random when the owner creates the account. In a deployed wallet it typically derives from a seed phrase via ZIP-32 hierarchical-deterministic derivation, but for this volume's purposes it is simply a uniform 256-bit string, $\mathsf{sk} \in \{0,1\}^{256}$. Every other key is a deterministic function of sk.

Deriving sub-secrets from one root requires a pseudorandom function. A *pseudorandom function* (PRF) is a keyed family $\{f_K\}_K$ whose outputs are, for a secret uniform key K, computationally indistinguishable from those of a truly random function (the *Crypto Guide*, §"Pseudorandom functions and permutations").

Definition 3.2 (Expansion function and reduction maps)**.** The *expansion function* $\mathsf{PRFexpand} : \{0,1\}^{256} \times \{0,1\}^{*} \to \{0,1\}^{512}$ is

$$\mathsf{PRFexpand}(\mathsf{sk}, t) := \text{BLAKE2b-512}\big(\texttt{``Zcash\_ExpandSeed''},\ \mathsf{sk} \,\|\, t\big),$$

unkeyed BLAKE2b-512 — a conventional, non-arithmetisation-friendly hash with a 512-bit (64-byte) digest (the *Crypto Guide*, §"PRFs from hash functions: length extension and keyed BLAKE2") — with the fixed 16-byte personalisation string `` ``Zcash_ExpandSeed'' `` and input $\mathsf{sk} \,\|\, t$. The protocol obtains its PRF not from BLAKE2b's native keyed mode but as *personalised, unkeyed* BLAKE2b over the concatenation $\mathsf{sk} \,\|\, t$: a PRF of t whenever sk is secret. The secret sk acts as the PRF key, and the leading bytes of t are a domain-separation tag identifying which sub-secret the call produces; we write the key as a subscript, $\mathsf{PRFexpand}_{\mathsf{sk}}(t)$, when convenient.

The 512-bit output is reduced into the required field or scalar set by one of two *reduction maps*, which read it as an integer and reduce modulo the relevant prime (p_{Pallas} and p_{Vesta} the Pallas base- and scalar-field orders of §2):

$$\begin{aligned} \text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Vesta}} \ \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Pallas}} \ \in \mathbb{F}_{p_{\mathsf{Pallas}}}, \end{aligned}$$

where LEOS2IP_{512} is the little-endian octet-string-to-integer map sending a 64-byte string to the integer $\sum_{i=0}^{63} x_i \, 256^i \in \{0, \ldots, 2^{512}-1\}$.

The width is deliberate. Because the value being reduced is 512 bits wide while each modulus is just over 2^{254} (of bit length 255), the modular reduction is statistically indistinguishable from uniform

on the target set: the modular bias is $O(2^{254}/2^{512}) = O(2^{-258})$, negligible. And the distinct tag bytes 0x06, 0x07, 0x08 below make the three derived outputs independent.

Construction 3.3 (Spend-side secrets and the spend validating key)**.** From sk derive the three spend-side secrets

$$\begin{aligned} \mathsf{ask} &:= \mathsf{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x06})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}} && \text{(the } \textit{spend authorising key}\text{)},\\ \mathsf{nk} &:= \mathsf{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x07})\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}} && \text{(the } \textit{nullifier key}\text{)},\\ \mathsf{r_{ivk}} &:= \mathsf{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\mathtt{0x08})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}} && \text{(the } \textit{CommitIvk randomness}\text{)}, \end{aligned}$$

so that ask and $\mathsf{r_{ivk}}$ are scalars in $\mathbb{F}_{p_{\mathsf{Vesta}}}$ (the scalar field of Pallas, i.e. the integers mod the Pallas group order p_{Vesta}), while nk is a base-field element in $\mathbb{F}_{p_{\mathsf{Pallas}}}$. The two targets differ only because the protocol uses nk as a field element — it keys the PRF at the heart of the derivation of the *nullifier*, the base-field element whose publication marks a note spent, constructed in §7 — whereas ask and $\mathsf{r_{ivk}}$ are scalars multiplying group elements.

Let $G^{\mathsf{SpendAuth}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\mathtt{z.cash:Orchard}, \mathtt{G}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$, a fixed, publicly-known generator of the Pallas group (a nothing-up-my-sleeve point, §2). The *spend validating key* is

$$\mathsf{ak} := [\mathsf{ask}]\, G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

Key generation canonicalises its sign: if $[\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$ has odd y-coordinate, ask is replaced by $-\mathsf{ask}$, so that ak always has even y and the x-coordinate $\mathsf{Extract}(\mathsf{ak})$ (§2) alone determines the point.

3.2 Viewing keys

Construction 3.4 (Viewing keys)**.** The *full viewing key* bundles the three quantities needed to recognise and decrypt one's own notes:

$$\mathsf{fvk} := (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathbb{F}_{p_{\mathsf{Vesta}}}.$$

From fvk derive

$$\begin{aligned} \mathsf{ivk} &:= \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}\big(\mathsf{Extract}(\mathsf{ak}),\ \mathsf{nk}\big) \in \{1, \ldots, p_{\mathsf{Pallas}} - 1\} \subset \mathbb{F}_{p_{\mathsf{Pallas}}},\\ K &:= \mathrm{LE}_{256}(\mathsf{r_{ivk}}) \in \{0,1\}^{256},\\ R &:= \mathsf{PRFexpand}\big(K,\ \mathtt{0x82} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})\big) \in \{0,1\}^{512},\\ (\mathsf{dk}, \mathsf{ovk}) &:= \big(R_{[0\,..\,256)},\ R_{[256\,..\,512)}\big), \end{aligned}$$

the *incoming viewing key* ivk (a base-field scalar), the *diversifier key* dk, and the *outgoing viewing key* ovk. Here $\mathsf{Commit}^{\mathsf{ivk}}$ is the named ShortCommit instance of §2; LE_{256} is the 256-bit little-endian bit encoding of §2, giving K the type PRFexpand requires of its key; $\mathsf{Extract}(\mathsf{ak})$ is the x-coordinate of ak, which after the sign canonicalisation of Construction 3.3 determines the point; and $\mathrm{I2LEOSP}_{256}$ denotes the 32-byte little-endian encoding of a field element or point.

Precision on the dk/ovk derivation, because the formula is easy to misread: a *single* 512-bit PRFexpand output R, keyed by $\mathsf{r_{ivk}}$ over the input $\mathtt{0x82} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk}$ (not over an empty message), is split into its two 256-bit halves — dk the first, ovk the second. The diversifier d of the next subsection is *not* part of this split; it derives afterwards from dk.

3.3 Diversified addresses

Construction 3.5 (Diversified addresses)**.** For any index $d_{\text{plain}} \in \{0,1\}^{88}$, define

$$
\begin{aligned}
d &:= \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(d_{\text{plain}}) \in \{0,1\}^{88} && \text{(the } \textit{diversifier}\text{)},\\
\mathsf{g_d} &:= \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd},\ d) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),\\
\mathsf{pk_d} &:= [\mathsf{ivk}]\,\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),
\end{aligned}
$$

where $\mathsf{FF1\text{-}AES256}_{\mathsf{dk}}$ is the NIST FF1 format-preserving encryption mode built on AES-256 (the *Crypto Guide*, §"Format-preserving encryption and FF1"), keyed by dk: a keyed *permutation* of the 88-bit index space, so each d_{plain} maps to a distinct 88-bit diversifier and different d_{plain} give unlinkable-looking addresses, all derived from the one key. The *diversified address* is the pair $\mathsf{addr} := (d, \mathsf{pk_d}) \in \{0,1\}^{88} \times \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$.

Choosing the index. The receiving wallet chooses the diversifier index freely: *any* of the 2^{88} values yields a valid, spendable address for the same ivk; no derivation fixes it and no index is forbidden — any 88-bit string is a valid diversifier index. The wallet allocates indices on demand, conventionally sequentially from $d_{\text{plain}} = 0$ (index 0 is the *default diversifier*), assigning a fresh index per payee or per invoice to keep addresses mutually unlinkable. A randomly drawn index would yield an equally valid address, but the specification states that diversifier indices SHOULD NOT be chosen at random: ZIP-32 specifies their usage in hierarchical-deterministic wallets, whose seed-only recovery of issued addresses depends on deterministic allocation.

Every index works for a reason specific to Orchard: the hash-to-curve $\mathsf{GroupHash}^{\mathsf{Pallas}}$ is total (§2) — it returns a point for every input — so every index yields a well-defined $\mathsf{g_d}$; in the negligible-probability event that the two summed SWU outputs (§2) cancel to the identity $\mathcal{O}$, the specification substitutes the fallback $\mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd}, \texttt{""})$, the same domain separator with the empty message (protocol specification § 5.4.1.6; crate `orchard`, `src/spec.rs`). This is a deliberate improvement over Sapling, the previous shielded generation, where the diversifier hash failed on roughly half of all indices and the wallet had to increment d_{plain} until it found a valid one; in Orchard no such rejection loop exists.

Rationale for routing the index through AES. The step from d_{plain} to d is a keyed *permutation* (format-preserving encryption) for three reasons that no simpler map satisfies together. (i) *Unlinkability:* wallets allocate indices in order $0, 1, 2, \ldots$, and consecutive diversifiers would reveal a shared owner and an issue count; encryption under dk scatters them pseudorandomly over $\{0,1\}^{88}$. (ii) *Bijection:* a permutation never collides two indices onto one d and is invertible — with dk the wallet recovers the index behind any of its addresses, storing no per-address secret; a plain hash is neither collision-free nor invertible. (iii) *Format preservation:* the address format fixes the diversifier at exactly 88 bits, and FF1 maps $\{0,1\}^{88} \to \{0,1\}^{88}$ bijectively, which a truncated hash cannot. The derivation runs *wallet-side*, never inside the proof circuit, so a conventional fast cipher (AES-256 under NIST FF1) is appropriate; arithmetisation-friendliness, the concern of §2, is irrelevant here.

3.4 The capability ladder

Figure 2 draws the hierarchy as it deserves to be read: as a deliberate capability ladder, each rung conferring strictly less power than the one above.

- *Spend* — sk / ask. Whoever holds ask can authorise spends; it sits at the top and nothing below it can recover it.
- *See everything* — fvk (hence ivk, ovk). Detects incoming notes (via ivk, which recovers $\mathsf{pk_d}$) and views outgoing notes (via ovk), but cannot spend, because deriving fvk from sk is one-way.

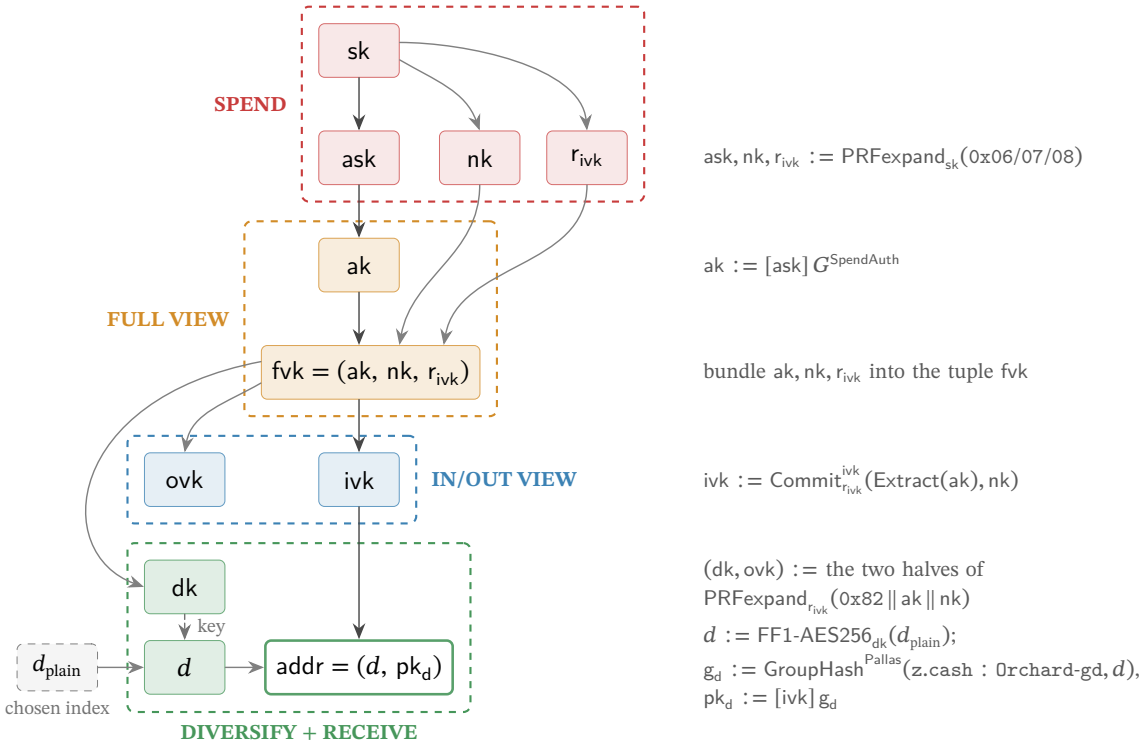

Figure 2: The Orchard key hierarchy as a capability ladder, read top to bottom. Each arrow is a derivation; the right-hand gutter shows the operation that produces the child from its parent(s). Not every edge is one-way: the bundling edges (ak, nk, r_{ivk} into the tuple fvk; d into the address) invert by projection, and the FF1 edge is a keyed permutation the dk holder inverts — the tiers are separated by the one-way steps sk → (ask, nk, r_{ivk}), ask → ak, fvk → (ivk, dk, ovk), and ivk → pk_d. The dk and ovk keys are the two halves of one $\mathsf{PRFexpand}_{r_{ivk}}$ output (tag 0x82, input ak||nk); they play different roles and land in different tiers — ovk is the *outgoing* viewing key, grouped with the *incoming* viewing key ivk as the IN/OUT viewing tier, whereas dk is the *diversifier* key, which sits in the address layer because it generates diversified addresses. Concretely d is the format-preserving encryption FF1-AES256 of a chosen index d_{plain} under the key dk (the dashed edge marks dk entering as the cipher key), and $\mathsf{pk}_d = [\mathsf{ivk}]\, \mathsf{g}_d$, so the address binds both keys. *Every node is secret key material except the chosen index d_{plain}, the diversifier d, and the diversified address (unshaded) — the address (d, pk_d), and with it d, is the only value the protocol ever publishes* (shading marks tier membership, not secrecy) — in particular the viewing keys fvk, ivk, ovk are *secret:* "viewing" names what they let the holder do, not that the protocol publishes them. Each dashed tier holds a strictly weaker capability than the one above, so a holder may receive a lower-tier key without gaining the powers above it. Drawn to match Constructions 3.3–3.5.

- *Detect incoming* — ivk alone. Sufficient to recognise notes addressed to the holder and scan the chain, nothing more — the key suitable for an always-online wallet server.
- *Receive* — a diversified address $(d, \mathsf{pk_d})$. A public handle anyone can pay; one ivk generates 2^{88} mutually unlinkable addresses — one per diversifier index (Construction 3.5), practically inexhaustible — all spendable by the same key.

Of these, only the diversified address is *public*. The viewing keys fvk, ivk, ovk are *secret* key material — "viewing" names what they let the holder *do* (see transactions), not that one may publish them. Disclosing a viewing key confers the corresponding visibility on the recipient — every payment to and from the wallet for fvk, incoming payments only for ivk, outgoing only for ovk — so the owner shares one only deliberately (with an auditor, say) and otherwise guards it like any other secret. The ladder is useful precisely because these capabilities are *separable*: one can delegate "see incoming" (ivk) without delegating "see everything" (fvk), and neither confers the ability to spend.

Two questions of motivation that the bare formulas do not answer. *Why is* ivk *a commitment,* $\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$*, rather than a plain hash?* Because ivk must both bind *and* hide. It binds ak (spend authority) and nk (nullifier derivation) to one identity, and the Action statement proves the linkage in-circuit by recomputing the commitment from the witnessed ak, nk, $\mathsf{r_{ivk}}$ (the CommitIvk check, §8) — but binding alone a collision-resistant hash would supply equally, and in-circuit recomputation works for any hash: the membership check of §8 recomputes the unblinded Sinsemilla Merkle-node hash $\mathsf{MerkleCRH}^{\mathsf{Orchard}}$ (§6), and Sapling even proved its BLAKE2s-based $\mathsf{CRH}^{\mathsf{ivk}}$ in-circuit. What the trapdoor $\mathsf{r_{ivk}}$ adds is *hiding*: the blinded $\mathsf{ShortCommit}$ is perfectly hiding (§2), so ivk — and hence every public $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ — reveals nothing about ak and nk, whereas the unblinded Sinsemilla hash is only collision-resistant (Theorem 2.2) with no one-wayness claim. Hiding is what keeps the IN/OUT tier strictly below FULL VIEW: an ivk holder, such as an always-online wallet server, cannot extract nk or ak. *Why is* nk *separate from* ask*?* So that the owner can delegate the capability to compute nullifiers — which a viewing key needs to determine which of the holder's notes are already spent (§7) — *without* conferring the ability to spend: the viewing key holds nk but never ask.

3.5 One ladder, both pools

The hierarchy just built carries one further property, worth stating at the source rather than discovering later: it is scoped to the *protocol*, not to a value pool. Orchard spending-key and viewing-key material grants authority to spend or view notes in *both* of the pools this volume follows — the legacy Orchard pool and the Ironwood pool (§1) — and a diversified address (a *receiver*, in the vocabulary of the ZIPs), with its corresponding ivk, is likewise scoped to the Orchard protocol, not to a pool (ZIP-229; ZIP-326). The entire ladder of this section therefore survives the pool split unchanged: an existing wallet's keys and addresses work in Ironwood exactly as they stand.

4 Minting: a note is born

The recipient of §3 now holds an address; a sender wishes to pay it. This stage answers the first requirement of §1, transfer of value (R1): value must change hands on a public chain that discloses neither payer, payee, nor amount. The vehicle is the *note*, and the first design problem is representational — the note must stay invisible on-chain yet remain provable by its owner. The protocol's answer is never to place the note on the chain at all: what the chain receives is only a *hiding commitment* to it.

4.1 The note and its commitment

Definition 4.1 (Orchard note)**.** An *Orchard note* — the private object representing one unit of value — is a tuple

$$\mathbf{n} := (\mathsf{addr},\ v,\ \rho,\ \psi,\ \mathsf{rcm}),$$

where $\mathsf{addr} = (d, \mathsf{pk_d})$ is the recipient's diversified address (Construction 3.5); $v \in \{0, \dots, 2^{64} - 1\}$ is the value in zatoshi; $\rho \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a per-note uniqueness seed, fixed at note creation (§7 constructs its provenance); $\psi \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is randomness the sender fixes through the note seed rseed, from which it derives deterministically (§4.2); and $\mathsf{rcm} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment trapdoor.

Construction 4.2 (Note commitment)**.** The protocol never publishes the note itself. What it records on-chain is the *note commitment*

$$\mathsf{cm} := \mathsf{NoteCommit}_{\mathsf{rcm}}\big(\mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi)\big) \ \in\ \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

where $\mathsf{NoteCommit}_{\mathsf{rcm}}$ is the Sinsemilla commitment $\mathsf{SinsemillaCommit}_{\mathsf{rcm}}$ under the domain `z.cash : Orchard-NoteCommit` (Definition 2.3). The chain stores the extracted coordinate

$$\mathsf{cmx} := \mathsf{Extract}(\mathsf{cm}) \ \in\ \mathbb{F}_{p_{\mathsf{Pallas}}},$$

the x-coordinate of the commitment point (Extract is constructed in §2.1), because the Merkle tree in which the commitment will come to rest (§6) consumes a field element, not a curve point.

The commitment is *perfectly hiding* — the chain learns nothing about addr, v, ρ, ψ from cm — and *computationally binding* — the sender cannot later claim the commitment was to a different note. These are precisely the two properties of a commitment scheme from the *Crypto Guide*, § "Hiding and binding", and §2.3 established both for the Sinsemilla commitment. The scheme is Sinsemilla for the reason given there: it is inexpensive to recompute *inside the circuit*, which the spender must one day do (§2.4).

The input encoding is worth a second look, because every byte of it recurs later in the section. The three address-related quantities are exactly the key material of §3: the diversifier $d \in \{0,1\}^{88}$, the diversified base $\mathsf{g_d} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd},\ d) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ — the per-address generator — and the *transmission key* $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ identifying the recipient (Construction 3.5); $\mathsf{addr} = (d, \mathsf{pk_d})$. Under the star encoding of §2.1, $\mathsf{g_d}^{\star}, \mathsf{pk_d}^{\star} \in \{0,1\}^{256}$. The NoteCommit input is the bit-string concatenation ($\|$) of these two point encodings with the little-endian integer encodings $\mathrm{LE}_{64}(v) \in \{0,1\}^{64}$ and $\mathrm{LE}_{255}(\rho), \mathrm{LE}_{255}(\psi) \in \{0,1\}^{255}$ (§2.1).

Why these five fields? The address and the value are the note's *content*: who is paid, and how much. The other three serve the machinery. The trapdoor rcm is what makes the commitment hiding (§2.3), while ρ and ψ exist solely to render the note's eventual *nullifier* — the token whose publication will one day mark this note spent — unique and unpredictable, a role that becomes apparent only when the note is spent (§7). A note carries randomness whose purpose is downstream.

4.2 The note seed: Ironwood derivations (lead byte `0x03`)

Definition 4.1 left two debts: the origins of ψ and rcm. The sender does not draw them independently; it draws a single uniform note seed $\mathsf{rseed} \in \{0,1\}^{256}$ (the key type PRFexpand requires, Definition 3.2) and derives all per-note randomness from it — alongside ψ and rcm, an *ephemeral secret key* esk, the sender's key half of the note encryption constructed in §5. The derivation is versioned: a note travels to its recipient as a *note plaintext* — the byte serialisation, also constructed in §5, that the sender encrypts to the address — and the plaintext opens with a *lead byte* naming its format. Which derivation applies is the lead byte's decision.

An Ironwood note is the same tuple $\mathbf{n} = (\mathsf{addr}, v, \rho, \psi, \mathsf{rcm})$ of Definition 4.1, committed by the same NoteCommit of Construction 4.2 (§4.1); ZIP-2005 (Proposed; activation bound to the NU6.3 network upgrade) carries the pool distinction on the note plaintext, through its lead byte. What changes is that byte — and, through it, the derivation of rcm.

The format is specified and deployed. ZIP-2005 § 4.7.3 gives the derivation, landed in protocol.tex at zips commit `69610984`, and the released `orchard` 0.15.0 — the version `librustzcash`, the deployed Zcash wallet library, pins — implements it: `enum NoteVersion` (`V2`/`V3`) carries the lead byte, `Note::from_parts` takes it as a fifth argument, and `Note::rcm()` dispatches on it (`src/note.rs`). The `bef8a27` development tree predates the release: its `Note::from_parts` (`src/note.rs:182`) still takes the four components (recipient, value, ρ, rseed) with no version tag, and its `Note::rcm()` is the legacy single-tag trapdoor of Remark 4.5.

Definition 4.3 (Ironwood note derivations)**.** For a lead-byte-`0x03` note with note seed rseed and uniqueness seed ρ (Definition 4.1), the sender derives, in this order,

$$\begin{aligned} \mathsf{esk} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x04} \,\|\, \mathrm{LE}_{256}(\rho))\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \psi &:= \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x09} \,\|\, \mathrm{LE}_{256}(\rho))\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}}, \\ \mathsf{rcm} &:= \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathsf{pre}_{\mathsf{rcm}})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \end{aligned}$$

where

$$\mathsf{pre}_{\mathsf{rcm}} := \mathtt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi),$$

each field in its canonical byte encoding — the points star-encoded (32 bytes each), v as 8 and ρ, ψ as 32 little-endian bytes — so $\mathsf{pre}_{\mathsf{rcm}}$ is a 137-byte string ($1+32+32+8+32+32$). In the rare event $\mathsf{esk} = 0$ the sender redraws a fresh rseed. (ZIP-2005 § 4.7.3 change; protocol.tex at zips commit `69610984`, where the hash is named $H^{\mathsf{rcm,Orchard}}_{\mathsf{rseed}}$.)

Two observations about Definition 4.3. First, the order is forced: rcm now comes last because its preimage includes ψ (ZIP-2005). Second, esk and ψ are byte-identical to their legacy lead-byte-`0x02` counterparts — only rcm changes (Remark 4.5) — and the same $\mathsf{g_d}^{\star}$, $\mathsf{pk_d}^{\star}$ encodings feed both $\mathsf{pre}_{\mathsf{rcm}}$ and the NoteCommit input of Construction 4.2: the trapdoor now commits, through PRFexpand, to the same material the commitment does. The purpose of this detour through BLAKE2b is *post-quantum binding*: every note field now traverses PRFexpand on its way into the trapdoor, so an adversary constrained to treat it as a random oracle cannot vary any field of a committed note without changing rcm — the property that makes every Ironwood-pool output note, and no Sapling- or Orchard-pool output note, a *recoverable* note, whose security analysis, recovery statement, and specified-but-undeployed qsk/qk key branch belong to §14.5 (ZIP-2005).

4.3 Lead bytes across pools and the legacy derivation

Which lead bytes a note plaintext may carry depends on the value pool and the block height — the lead byte is consensus-visible format versioning, not a free choice.

Definition 4.4 (Allowed lead bytes, pool-parameterised)**.** Let pool be the chain value pool of a note received at block height h — SaplingPool, the value pool of the preceding Sapling shielded generation, or OrchardPool or IronwoodPool, the formal names for the Orchard and Ironwood pools of §1 — and let h_{Canopy} be the activation height of the Canopy network upgrade, which deployed ZIP-212's rseed-derived note-plaintext format and its `0x02` lead byte; the constant ZIP212GracePeriod is the fixed length, set by ZIP-212, of the window after h_{Canopy} in which both formats remain valid.

Then

$$\mathsf{allowedLeadBytes}^{\mathsf{pool}}(h) := \begin{cases} \{\texttt{0x01}\} & h < h_{\text{Canopy}}, \\ \{\texttt{0x01}, \texttt{0x02}\} & h_{\text{Canopy}} \le h < h_{\text{Canopy}} + \mathsf{ZIP212GracePeriod}, \\ \{\texttt{0x02}\} & \text{afterwards, } \mathsf{pool} \neq \mathsf{IronwoodPool}, \\ \{\texttt{0x03}\} & \text{afterwards, } \mathsf{pool} = \mathsf{IronwoodPool}. \end{cases}$$

For an Ironwood-pool note plaintext the only allowed lead byte is `0x03` (ZIP-2005 § 3.2.1 change; protocol.tex at zips commit `69610984`).

Coinbase outputs — the outputs of the transaction that mints each block's subsidy — obey the same discipline: an Ironwood coinbase output must carry lead byte `0x03` (ZIP-258), the Ironwood analogue of the Sapling/Orchard rule that coinbase outputs use the post-ZIP-212 `0x02` format (ZIP-213).

Remark 4.5 (The legacy derivation)**.** Lead-byte-`0x02` notes — all Orchard-pool notes, before and after NU6.3 — derive

$$\mathsf{rcm} := \mathrm{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x05} \,\|\, \mathrm{LE}_{256}(\rho))\big),$$

with esk and ψ exactly as in Definition 4.3. The specification packages both derivations under one dispatcher, $\mathrm{Derive}^{\mathsf{Orchard}}_{\mathsf{rcm},\,\mathsf{rseed}}(\mathsf{leadByte}, \dots)$, selecting $\texttt{0x05} \,\|\, \mathrm{LE}_{256}(\rho)$ for `0x02` and the hashed $\mathsf{pre}_{\mathsf{rcm}}$ for `0x03` (ZIP-2005 § 4.7.3 change). The dev-tree `Note::rcm()` at `bef8a27` implements only this legacy branch (`src/note.rs:132-136`), with `PrfExpand::ORCHARD_RCM` = `0x05` (zcash_spec 0.2.1, `src/prf_expand.rs:88`), no version dispatch, and no `0x03`/hashed-$\mathsf{pre}_{\mathsf{rcm}}$ branch; the released `orchard` 0.15.0 that `librustzcash` pins adds the `NoteVersion` dispatch — `V2` to `rcm_v2`, `V3` to `rcm_v3` over the `0x0B`-separated $\mathsf{pre}_{\mathsf{rcm}}$ (`src/note.rs`) — deploying the derivation of Definition 4.3.

5 Delivery: the note reaches its owner

The mint succeeded too well. The on-chain cmx (§4) conceals the note entirely, leaving the question of how the recipient learns of the note with which they were paid. The sender must deliver the note to them, privately, over the same public chain. Orchard achieves this with *in-band secret distribution*: every Action carries, beside its commitment, a ciphertext only the recipient can open. This section constructs the two ciphertexts the sender attaches, the trial decryption by which the recipient finds and verifies the note, and the synchronisation procedure — a wallet restoring from a seed — that the same machinery supports.

5.1 Sender: one-shot Diffie–Hellman and the AEAD

Alice holds Bob's address $(d, \mathsf{pk_d})$, with

$$\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}, \qquad \mathsf{g_d} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd},\ d)$$

(Construction 3.5, §3). She forms the *note plaintext*

$$\mathsf{np} := (\texttt{0x03},\ d,\ v,\ \mathsf{rseed},\ \mathsf{memo}),$$

where `0x03` is the Ironwood note-plaintext lead byte, the 32-byte note seed rseed — drawn uniformly at random, fresh for each output note — is the master randomness from which the note's rcm and ψ were derived at minting (Definition 4.3, §4) and from which the ephemeral secret esk below derives as well, and memo is a fixed 512-byte field of sender-chosen data. Each of these derivations is one PRFexpand call keyed by rseed and distinguished by a leading domain-separator byte (the one derivation display, Definition 4.3); esk and ψ take as input $\rho := \mathsf{nf_{old}}$, encoded as 32 little-endian bytes — which binds the note's secret randomness to its Action, $\mathsf{nf_{old}}$ being the base-field nullifier of the note spent alongside (constructed in §7) — and rcm, derived last, takes every note field.

Construction 5.1 (Note encryption)**.** Alice runs a one-shot Diffie–Hellman key agreement to the diversified base (the *Crypto Guide*, §"The Diffie–Hellman protocol" and §"Static and ephemeral keys"), yielding a *fresh* key:

$$\mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}, \qquad \mathsf{s} := [\mathsf{esk}]\,\mathsf{pk_d}, \qquad K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s},\ \mathsf{epk}),$$

with epk the *ephemeral public key*. She then seals the plaintext under an AEAD — authenticated encryption with associated data, instantiated by ChaCha20-Poly1305 (the *Crypto Guide*, §"The ChaCha20-Poly1305 AEAD"); we write $\mathsf{Enc}_K/\mathsf{Dec}_K$ for its encryption and decryption under the key K:

$$C_{\mathrm{enc}} := \mathsf{Enc}_{K_{\mathrm{enc}}}(\mathsf{np}).$$

The key-derivation function KDF (the *Crypto Guide*, §"Key-derivation functions") is BLAKE2b-256 with personalisation `Zcash_OrchardKDF` over $\mathsf{s}^\star \,\|\, \mathsf{epk}^\star$, where $\star$ is the point encoding of §2.

Why exactly the intended recipient? Because $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, Bob can reach the same shared secret from his side as

$$[\mathsf{ivk}]\,\mathsf{epk} \;=\; [\mathsf{ivk}][\mathsf{esk}]\,\mathsf{g_d} \;=\; [\mathsf{esk}]\,\mathsf{pk_d} \;=\; \mathsf{s},$$

so, beside the sender (who knows esk), exactly the holder of ivk can recover K_{enc} from the published pair $(\mathsf{epk}, C_{\mathrm{enc}})$.

5.2 The outgoing ciphertext

One ciphertext serves the recipient; a second serves the sender's own books. The pair $(\mathsf{pk_d}, \mathsf{esk})$ is sealed under the *outgoing cipher key*,

$$\mathsf{ock} := \text{BLAKE2b-256}\big(\texttt{Zcash\_Orchardock},\ \mathsf{ovk} \,\|\, \mathsf{cv}^{\mathsf{net}} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk}\big), \qquad C_{\mathrm{out}} := \mathsf{Enc}_{\mathsf{ock}}(\mathsf{pk_d},\ \mathsf{esk}),$$

points in $\star$-encoding, where $\mathsf{cv}^{\mathsf{net}}$ is the Action's net value commitment — a Pallas-point commitment to the Action's value change, constructed in §7 — and ovk is the outgoing viewing key of Construction 3.4. The ciphertext C_{out} carries $(\mathsf{pk_d}, \mathsf{esk})$ so that the *sender* — or an auditor holding ovk — can re-derive $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d}$ and read the note as well. Because ock binds the Action's own $\mathsf{cv}^{\mathsf{net}}$, cmx, and epk, a C_{out} cannot be transplanted onto another Action.

The Action publishes

$$(\mathsf{epk},\ C_{\mathrm{enc}},\ C_{\mathrm{out}}) \qquad \text{alongside} \qquad \mathsf{cv}^{\mathsf{net}},\ \mathsf{nf},\ \mathsf{rk},\ \mathsf{cmx},$$

with nf the Action's base-field nullifier and rk its re-randomised validating key — both, like $\mathsf{cv}^{\mathsf{net}}$, public fields of the Action constructed in §7.

5.3 Recipient: trial decryption and re-derivation

Nothing on chain marks which Action pays Bob, so his wallet *trial-decrypts every* Orchard-protocol output: for each it recomputes

$$\mathsf{s} := [\mathsf{ivk}]\,\mathsf{epk}, \qquad K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s},\ \mathsf{epk}), \qquad \mathsf{np} := \mathsf{Dec}_{K_{\mathrm{enc}}}(C_{\mathrm{enc}}).$$

The AEAD tag makes an incorrect guess fail cleanly (the decryption returns $\perp$), so a successful open *is* the detection that the note was his.

After a successful open Bob re-derives the full note from (d, v, rseed) — taking $\rho := \mathsf{nf_{old}}$ from the same Action — and performs the two checks that close the obvious forgeries: (i) that the sender formed the ephemeral key honestly, $[\mathsf{esk}]\,\mathsf{g_d} = \mathsf{epk}$ for the esk re-derived from rseed (the ZIP-212

check, defeating a mauled epk); and (ii) that the note is indeed the one the Action committed on chain,

$$\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)\big) = \mathsf{cmx}.$$

Only when both hold does he accept. A sender can therefore never make Bob accept a note inconsistent with the commitment.

The re-derivation fixes an order. Because the lead-byte-0x03 trapdoor rcm depends on $\mathsf{g_d}$, $\mathsf{pk_d}$, and ψ (Definition 4.3), the specification reorders both the ivk and the fvk decryption procedures: recover $\mathsf{g_d}$ and $\mathsf{pk_d}$ first, then ψ, then rcm (ZIP-2005, §§ 4.20.2–4.20.3 changes). The routing is deployed: the released `orchard` 0.15.0 routes the two lead bytes to two note-encryption domains, `OrchardDomain` (0x02) and `IronwoodDomain` (0x03) (`src/note_encryption.rs`), which `zcash_client_backend` scanning consumes (`src/scanning.rs`); the `orchard` `bef8a27` development tree still carries a single 0x02-only `OrchardDomain`. And because keys are scoped to the protocol rather than to a pool (§3.5), the same ivk trial-decrypts both pools — one key, two domains (ZIP-326; `zcash_client_backend` `src/scanning.rs`).

5.4 Synchronisation, including from cold storage

Trial decryption scales from one payment to the whole chain: it is the operation a wallet performs when it restores from a seed. Restoring, the wallet walks the chain forward from the account's "birthday" height and trial-decrypts every Action. A *light wallet* — one that delegates block storage to a server it need not trust with keys — does so against *compact blocks*, blocks stripped to the fields trial decryption needs. Each compact output carries only the first 52 bytes of C_{enc} — those covering the *compact note plaintext*: the lead byte (1 byte; 0x03 for Ironwood outputs, the only allowed value in that pool — 0x02 appears only when scanning legacy Orchard-pool outputs), d (11 bytes), v (8 bytes), and rseed (32 bytes) — together with epk, cmx, and the Action's nullifier nf, which the light wallet needs twice: it supplies $\rho := \mathsf{nf}_{\text{old}}$ for the cmx recomputation that detects the note, and it is the source of the chain's nullifier set (§7) against which spentness is checked below (ZIP-307; `zcash_client_backend` `proto/compact_formats.proto`).

The wallet records each note that opens *with its leaf position* — the index at which its cmx was appended, read directly off the block — and that position is precisely what the commitment tree consumes: the wallet marks the position in its shard tree and can then build the note's authentication path on demand, by root-only assembly, without replaying every commitment (§6 constructs the tree, the shard store, and this assembly). To determine which recovered notes remain spendable it additionally requires the nullifier key nk (§3): it computes each note's nullifier (§7) and discards those whose nullifier already appears in the chain's nullifier set. The end state is a set of unspent notes, each with a value, a position, an authentication path, and a nullifier — exactly what the Action prover of §8 consumes.

The light-client surface follows the pool split. Compact blocks gain an `ironwoodActions` list (the same `CompactOrchardAction` message) and an `ironwoodCommitmentTreeSize`, and the synchronisation just described runs per pool, against that pool's own commitment tree (`zcash_client_backend` `proto/compact_formats.proto`; §6). The note has reached its owner, who can detect it, verify it, and place it in the tree; there it now rests.

6 Resting: the note in the commitment tree

Minted (§4) and delivered (§5), the note now rests, and the journey reaches its second requirement (R2, §1): when the note is eventually spent, its owner must prove it *valid* — that someone actually created it — without revealing which note it is, for otherwise Alice could spend a note she invented. On a transparent chain this is trivial: the output being spent is present in the UTXO set (the set of unspent transparent outputs) that every node maintains — but pointing at the output would deanonymise the payment. The shielded solution is this section's object. The network maintains one append-only

Merkle tree — a binary hash tree, each parent a hash of its two children, so the single root digests every leaf — whose leaves are every note commitment ever published, in creation order; Alice proves *membership*, by a Merkle authentication path from her leaf cm_x to a root the network trusts. Inside the circuit the path is a private witness (the membership check the Action statement imposes, §8), so the verifier learns that her note is one of the leaves but not which one: the anonymity set is the entire tree.

6.1 The tree, its hash, and its anchors

Definition 6.1 (Orchard note commitment tree)**.** The *note commitment tree* is an append-only Merkle tree of fixed depth 32 — room for 2^{32} notes — whose leaves are the commitments cm_x (§4) in creation order; positions not yet used hold the *uncommitted* sentinel value 2. Number the layers from the root, layer 0, down to the leaves, layer 32. The parent at layer ℓ of a left child l and right child r at layer $\ell + 1$ is the layer-tagged Sinsemilla short hash

$$\mathsf{MerkleCRH}^{\mathsf{Orchard}}_{\ell}(l, r) := \mathsf{ShortHash}_{\texttt{z.cash:Orchard-MerkleCRH}}\big(\mathrm{LE}_{10}(31-\ell) \,\|\, \mathrm{LE}_{255}(l) \,\|\, \mathrm{LE}_{255}(r)\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$

the unkeyed short form of Definition 2.3 over the $10 + 255 + 255 = 520$-bit message (52 ten-bit Sinsemilla chunks), under the same little-endian input convention as $\mathsf{Commit}^{\mathsf{ivk}}$ and $\mathsf{NoteCommit}$. The ten-bit tag is the fold *height* counted from the leaves, $31-\ell$: 0 for the leaf-level combine, 31 for the combine that yields the root (protocol specification § 5.4.1.3, $\mathsf{MerkleCRH}^{\mathsf{Orchard}}$; crate `orchard`, `src/tree.rs`). Leaves, internal nodes, and the root all live in $\mathbb{F}_{p_{\mathsf{Pallas}}}$. Appending a block's note commitments yields a new root: the root of the tree as it stands after block h is the *anchor* of block h, a single field element, so every main-chain block defines exactly one anchor. A spend proves membership against the anchor of any main-chain block, and all Actions of one bundle prove against a single shared anchor (§8).

Every node of the tree is a single base-field element by necessity, not convention. The spender recomputes the hash layer by layer *inside* the Action circuit to prove membership (the membership check, §8), and that circuit is arithmetic over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (§2): every wire carries one such field element. A curve-point node would carry two coordinates and an on-curve constraint at each of the 32 layers; reducing each node to its x-coordinate halves the path data the circuit threads and removes that bookkeeping. This is precisely why $\mathsf{MerkleCRH}$ ends in $\mathsf{Extract}$: Sinsemilla yields a point, and $\mathsf{Extract}$ returns it to the field, so that each layer's output is directly the next layer's input and the one hash composes all the way to the root. Dropping the y-coordinate at every node is harmless for the reason §2 gave for $\mathsf{Extract}$ itself: the tree uses each node only as a binding digest, never to recover a child point, and a collision in the x-coordinate alone yields either a collision in the underlying Sinsemilla hash or a pair of *opposite* points ($\mathsf{Extract}$ identifies P with $-P$, §2.1) — both non-trivial discrete-log relations, the second by one extra case; §12 discharges the two together alongside Theorem 2.2. The layer tag, finally, is replay protection: because the tag fixes the fold height, a hash computed at one height cannot be replayed at another.

Nor is the sentinel arbitrary. At $x = 2$ the Pallas curve equation $y^2 = x^3 + 5$ (the *Math Guide*, §"Elliptic curves") gives $y^2 = 2^3 + 5 = 13$, and 13 is a non-square modulo both Pasta primes (verified in the *Math Guide*, §"Number theory for cryptography"), so 2 is not the x-coordinate of any Pallas point: it can never equal a valid cm_x, and an empty position can never be silently confused with a genuine commitment.

6.2 The authentication path from public data

Construction 6.2 (Authentication path). Alice's commitment occupies a fixed *position* pos — its index in creation order — which her wallet records the moment trial decryption succeeds (§5). Fix an anchor, that of any block at or after the one that mined her note, and let n be the number of leaves in that block's tree. Write $\text{pos} = b_{31} \cdots b_1 b_0$ in binary, and count levels $i = 0, \ldots, 31$ from the leaf upward (the level-i fold applies the layer-$(31-i)$ hash, whose tag is therefore i, Definition 6.1). The level-i ancestor of her leaf is a left child when $b_i = 0$ and a right child when $b_i = 1$, and the path's level-i entry is that ancestor's *sibling* in the tree of size n — one node per level, 32 in all. The membership check (§8) reverses the construction: starting from cmx it folds in one path entry per level — placing the running node on the side b_i dictates, left when $b_i = 0$, right when $b_i = 1$, and the sibling opposite — and asserts that the final value equals the public anchor; the bits b_i and the siblings stay inside the witness.

Each path entry is one of two kinds, and the difference organises everything a wallet does. When $b_i = 1$ the sibling subtree lies to the *left* of her leaf: it spans commitments older than hers, so every position in it is filled — a complete subtree — and, because the tree appends only on the right, its root is final, identical at every anchor. When $b_i = 0$ the sibling subtree lies to her *right* and its value depends on the anchor: its root hashes the commitments it holds at size n, unfilled positions taking the sentinel 2; a sibling subtree still entirely empty at size n contributes a per-level *constant*, the sentinel folded up through the layer-tagged hash. The path is therefore a deterministic function of pos and the first n public leaves, and of nothing else; the only secret is which leaf is hers. Figure 3 draws the computation at depth 3.

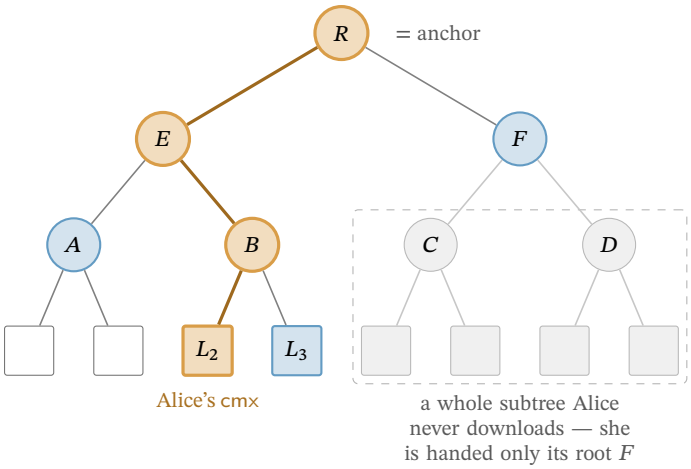

Figure 3: Computation of a Merkle authentication path (drawn at depth 3; Orchard's tree has depth 32). The leaves are the public note commitments cmx, in order; Alice's note sits at leaf L_2 (highlighted). To prove membership she computes the authentication path of L_2 — the siblings L_3, A, F (shaded) — then recomputes the bold route from L_2 up through B and E to the root R, the anchor, folding in the sibling hash at each level. Efficiency is the only reason not to hash the whole tree: a far subtree (dashed box) enters her path solely through its single root, here F, so she never downloads its leaves (faded) — the GetSubtreeRoots shortcut (§6.3) that lets even a freshly restored wallet witness a years-old note.

6.3 The wallet's witness: shardtree, shards, and the cap

In practice the wallet neither rebuilds the path from the leaves at each spend nor retains the whole tree. As it scans the chain (§5) it appends every published cmx — its own and everyone else's — but keeps only what a future path needs: the nodes from each of its own notes up to the root, the complete-subtree roots flanking those paths, and one checkpoint per block; the rest it prunes. This is the design

of the `shardtree` crate (`shardtree/src/store.rs`) that `zcash_client_backend` drives. The wallet marks its note's position the moment trial decryption succeeds; a *checkpoint* stores merely the position of that block's last leaf — equivalently the leaf count n, the size that fixes the block's anchor — and the anchor itself is recomputed from the retained nodes on demand, never stored. To produce the path for a spend against a chosen block, the wallet descends its retained tree to the marked leaf and, on the way back to the root, reads off each sibling subtree's root: a complete subtree contributes its stored root, a subtree lying wholly beyond the leaf count n contributes the per-level empty root, and the single subtree straddling position n is rehashed from its retained nodes. The leaf count enters only as this cutoff — every position $\geq n$ counts as empty — so one retained tree reproduces the path against any block it has checkpointed.

Nothing in Construction 6.2 requires Alice to have been online: its inputs are public, so a freshly restored wallet — a seed phrase and nothing else — rebuilds the same state by replaying the chain's note commitments. Nor need it touch all 2^{32} positions. Cut the tree at half its depth. Below the cut lie the *shards*, the height-16 subtrees — 2^{16} leaves apiece, up to 2^{16} of them — and above it a height-16 *cap* whose own leaves are the shard roots. (The backend pins the shard height to half the tree depth, $32/2 = 16$: constant `ORCHARD_SHARD_HEIGHT`, `zcash_client_backend/src/data_api.rs`.) A leaf's 32 siblings divide along the same cut: the low 16, levels 0 through 15, lie inside its own shard, and the high 16, levels 16 through 31, are cap nodes, each the root of a subtree spanning whole shards. The two halves reach Alice by entirely different routes, and that asymmetry is the economy of the scheme.

Her own shard she must hash from leaves: its 16 low siblings are the within-shard subtrees of the path descent — complete to her left, straddling or empty to her right — so she scans that shard's commitments block by block, her own and everyone else's, and folds them; this is the one subtree whose 2^{16} leaves she handles individually. Every other shard she takes as a single 32-byte root (the dashed far subtree of Figure 3): the cap's leaves are exactly those roots, which a light-client server streams one per completed shard (the GetSubtreeRoots call of the light-client protocol, §5; `zcash_client_backend`, `proto/service.proto`); she folds them upward with the same layer-tagged MerkleCRH, one application per cap level — the upper half of the tree's 32 layers — and reads her high siblings off the result. A shard that does not yet exist (the empty right of the cap) contributes the per-level empty constant rather than a download, and her own shard she holds from leaves whether or not it has completed, so she never needs its root from the server. Her level-16 sibling is thus a neighbouring shard's root, and each sibling above it a fold of $2, 4, 8, \ldots$ roots she already holds.

Example 6.3 (Witnessing the newest shard)**.** Let Alice's note be the last leaf of the newest shard, shard k. Below the cut, all 16 low siblings are complete left subtrees, tiling the shard's earlier $2^{16} - 1$ positions, so she ingests essentially the whole shard. Above the cut, the cap's right side is empty — no shard succeeds k, so those siblings are the empty constant — while its left side folds the roots of shards $0, \ldots, k-1$, each supplied by GetSubtreeRoots. Stitching the halves — her leaf folds up to the shard-k root, which folds up the cap to the anchor — gives the full 32-level path.

The 2^{16} throughout bounds the cap's *capacity* — 2^{32} leaves over 2^{16} per shard — not Alice's traffic. She fetches one root per shard that actually exists, $\lceil n/2^{16} \rceil$ of them at leaf count n: a few hundred at present tree sizes, growing by one every 2^{16} outputs. The whole history above her shard thus arrives as a few kilobytes of roots in place of millions of leaves. And she performs the fold locally, never asking a server for one particular note's path — which would reveal which note is hers; the finished path is the private witness of the membership check (§8), and only the anchor is public.

6.4 Anchor choice, reorgs, and the anonymity set

By the time Alice's transaction reaches a block, the live tree has advanced past her chosen anchor, and this costs nothing: appending leaves creates new roots and alters no old one, so the anchor still names one fixed historical tree, her path still folds to exactly that root, and the membership check asserts exactly that equality — collision resistance of the layer-tagged hash (Theorem 2.2) makes a

path to that root for a non-member infeasible. Consensus is equally generous: it accepts the anchor of *any* main-chain block from Orchard activation (the NU5 network upgrade) onward — this for the legacy Orchard-pool tree; the Ironwood tree's anchors run from its own activation (§6.5). In neither pool is there a sliding window of admissible anchors, so proving is fully decoupled from broadcasting: Alice need not race the chain tip.

The one recency caveat runs in the opposite direction. A reorganisation — the replacement of the chain's most recent blocks by a competing branch (the *Consensus Guide*, §"Reorg rails, maturity, and the finality floor") — can strike the anchoring block from the main chain, invalidating the anchor and forcing Alice to rebuild the transaction; deeper anchors make the event rarer, but no depth rules it out. The often-cited 100-block figure is the specification's description of the reorganisation depth nodes are expected to handle (protocol specification § 3.3); the deployed validator, Zebra, actually rolls back automatically to a depth of at most 1000 blocks (constant `MAX_BLOCK_REORG_HEIGHT`, crate `zebra-chain`, `src/parameters/constants.rs`). Both figures are node policy rather than consensus, as the *Consensus Guide* section just cited develops.

Deployed wallets nevertheless anchor close to the tip, by deliberate choice rather than necessity. The anchor's tree is the anonymity set in which the membership check conceals the spent note — the proof reveals only that the note is *some* leaf committed by that anchor — so the most recent admissible anchor, naming the largest such tree, maximises that set, while an idiosyncratically old or deep anchor would instead fingerprint the spender. The wallet backend accordingly selects the most recent block it has checkpointed lying at least a fixed number of confirmations behind the tip — by default, per ZIP-315, 3 confirmations for notes the wallet produced itself and 10 for notes received from third parties (type `ConfirmationsPolicy`, `zcash_client_backend/src/data_api/wallet.rs`) — trading a sliver of reorg robustness for set size and uniformity. Consensus mandates none of this; it is wallet policy. Nor is an old anchor a soundness hole: membership in an older, smaller tree implies the note was committed even earlier, which is precisely what Alice must establish — preventing a *second* spend of the same note is not the anchor's job but the nullifier's, the base-field element published with every spend and constructed in §7.

6.5 Two pools, two trees

The volume's two pools (§1) are distinguished by *state*, not identity. Each keeps its own, fully independent note commitment tree and nullifier set (ZIP-229), hence its own anchors, and its own chain value pool balance — the consensus-tracked net value the pool holds. The Ironwood tree is built by the identical depth-32 $\mathsf{MerkleCRH}^{\mathsf{Orchard}}$ construction of Definition 6.1, capacity 2^{32} (ZIP-258); it starts *empty* at the pool's activation (the NU6.3 network upgrade), and an Ironwood Action proves membership against $\mathsf{anchor}_{\mathsf{Ironwood}}$, the anchor of the Ironwood tree, admissible from that activation onward. The Orchard tree, meanwhile, keeps growing after the split: every Orchard-pool Action still appends a cm_{x}, because spends under the *cross-address restriction* — the post-NU6.3 consensus rule confining each Orchard-pool Action's created note to the spent note's own receiver — are paired with fabricated or *dummy* zero-valued output notes (rule, fabrication, and the dummy treatment are all taken up in §10), and its anchors remain live for spend proofs. The wallet machinery of §6.3 carries over unchanged — the backend pins `IRONWOOD_SHARD_HEIGHT` to the same 16, in the same file, as its Orchard twin. Where a note rests, and against which tree it may be spent, are henceforth facts about *which pool* — state the rest of the journey must thread through every proof.

7 Spending: the nullifier, the balance, and the signatures

The note has rested long enough; its owner now spends it, and the two remaining requirements of §1 come due at once: R3, that no note can be spent twice, and R4, that no transaction creates value — each to be achieved while disclosing neither which note is spent nor what it is worth. This section constructs the machinery the spend contributes to an Action (the unit of shielded transfer named in

§1: one Action spends one old note and creates one new one; §8 assembles it in full): the nullifier that retires the old note, the value commitment and binding signature that prove conservation, the transaction digest that both signatures sign, and the re-randomised signature that proves spend authority. Each security property is stated where its object is constructed and proved once in §12, the volume's standing division of labour.

7.1 The nullifier, and the loop back to minting

The third requirement: to prevent the owner from spending the same note twice, without a public registry of spent notes — which would leak exactly what is being hidden. Each note yields a single deterministic tag, its *nullifier*, that appears only when its owner spends the note; the network maintains a set of all revealed nullifiers and rejects any repeat. The nullifier must satisfy two properties that pull in opposite directions: it must be *deterministic* (the same note always yields the same tag, making a second spend detectable) yet *unlinkable* (publishing it reveals nothing about which note or address it came from).

Definition 7.1 (Orchard nullifier)**.** For a note **n** with commitment cm (§4) and the owner's nullifier key nk (§3),

$$\mathsf{nf} := \mathsf{Extract}\big(\big[\, F_{\mathsf{nk}}(\rho) + \psi \,\big] G^{\mathsf{nf}} + \mathsf{cm}\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$

where $F_{\mathsf{nk}} : \mathbb{F}_{p_{\mathsf{Pallas}}} \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a circuit-efficient PRF — Poseidon over the Pallas base field, keyed by nk (the *Crypto Guide*, §"Arithmetisation-friendly hashing: Poseidon"); $G^{\mathsf{nf}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard}, \texttt{K}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ is a fixed nothing-up-my-sleeve Pallas generator (§2.2); the scalar $F_{\mathsf{nk}}(\rho) + \psi$ lives in $\mathbb{F}_{p_{\mathsf{Pallas}}}$, a valid Pallas scalar because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$; and $\mathsf{Extract}$ takes the x-coordinate (§2.1). We write G^{nf} for this base to distinguish it from the spend-auth generator $G^{\mathsf{SpendAuth}}$ of §3.

We read the construction against the two requirements. *Determinism*: every ingredient (nk, ρ, ψ, cm) is fixed once the note exists, so nf is a fixed function of the note — spending it twice yields the same nf twice, and the network rejects the second. *Unlinkability*: the term $[F_{\mathsf{nk}}(\rho)+\psi]\,G^{\mathsf{nf}}$ masks the commitment with a value that appears random to anyone without nk, so no one can tie the published nf back to cm or to the address — this is where the note's ρ and ψ, carried since §4, perform their function. Three security properties sharpen this reading:

- *No double-spend* (the *Orchard Book* calls this *balance*, since double-spending is how one would create value): each note has exactly one nullifier, and forging a colliding nullifier for a different note reduces to the discrete-logarithm problem on Pallas.
- *Spend unlinkability*: without nk, the published nf is unlinkable to the note or address, under the Decisional Diffie–Hellman (DDH) assumption on Pallas — that $([a]G, [b]G, [ab]G)$ is computationally indistinguishable from $([a]G, [b]G, [c]G)$ for independent uniform a, b, c (the *Crypto Guide*, §"Diffie–Hellman: computational and decisional") — *or*, independently, the PRF assumption on F (§3): either assumption alone suffices. The disjunction is the design rationale for the nullifier's group structure — defence in depth, unlinkability surviving the failure of one assumption (the *Orchard Book*, `design/nullifiers.md`).
- *Forward consistency (Faerie resistance)*: a freshly created note takes as its uniqueness seed the nullifier of the note spent alongside it, $\rho_{\mathsf{new}} := \mathsf{nf}_{\mathsf{old}}$; since the network already guarantees nullifiers are unique, this forces every note's ρ to be globally unique, closing an attack from earlier designs in which forged notes could be made to share a nullifier.

The last item closes the loop opened at minting: the ρ that §4 could only type as a base-field element fixed at note creation now acquires its provenance. Every note is born from a spend, and

inherits as ρ the nullifier that spend reveals — a globally unique value by the very double-spend rule this section establishes.

> **Proposition 7.2** (Nullifier uniqueness)**.** *Each note determines exactly one nullifier, and any efficient adversary that produces two distinct notes sharing a nullifier yields a solver for the discrete-logarithm problem on Pallas (modelling the derivation of G^{nf} as a random oracle, §2.2).*

The proposition is proved in §12; here we only record what the construction is asked to deliver. The unlinkability claim above is a design-level guarantee, not a deferred theorem: it enters the volume's privacy statement as one assumption of the informal end-to-end theorem that closes §12.

Both pools of this volume use this derivation unchanged: an Ironwood note's nullifier is Definition 7.1 verbatim, and a note's nullifier is checked only against its *own* pool's nullifier set — the two pools keep separate, independent nullifier sets (ZIP-229; the split's state is laid out in §9).

7.2 Conservation without disclosure: value commitments and the binding signature

The fourth requirement: to guarantee that a transaction creates no value, while concealing every amount. The instrument is the additive homomorphism of Pedersen commitments, constructed in the *Crypto Guide*, §"The Pedersen commitment": commit to each amount, and verify that the commitments *sum* to the publicly declared net flow — the individual values remain hidden, while their balance is publicly verifiable.

> **Definition 7.3** (Net value commitment)**.** Each Action commits to its own value change with a Pedersen commitment
>
> $$\mathsf{cv}^{\mathsf{net}} := [\, v_{\mathrm{old}} - v_{\mathrm{new}} \,]\, V^{\mathsf{Orchard}} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$
>
> where v_{old} is the spent value, v_{new} the created value, $\mathsf{rcv} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ the commitment randomness (a Pallas scalar), and
>
> $$V^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"v"}),$$
> $$R^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"r"}) \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$$
>
> are nothing-up-my-sleeve points (§2.2), hence independent Pallas generators.

The spender draws each rcv uniformly at random and afresh for every Action, at the moment of bundle assembly. In contrast to the note's rcm, ψ, and esk — all derived deterministically from the note seed rseed (§4) — the trapdoor rcv belongs to no note and is never transmitted; its uniformity and secrecy are precisely what make $\mathsf{cv}^{\mathsf{net}}$ hiding, and what keep the aggregate bsk introduced next known only to the transaction author.

We now apply the homomorphism. Summing over the m Actions of a transaction,

$$\sum_{i=1}^{m} \mathsf{cv}^{\mathsf{net},i} = \Big[\textstyle\sum_i (v_{\mathrm{old},i} - v_{\mathrm{new},i})\Big] V^{\mathsf{Orchard}} + [\mathsf{bsk}]\, R^{\mathsf{Orchard}}, \qquad \mathsf{bsk} := \textstyle\sum_i \mathsf{rcv}_i \;\in\; \mathbb{F}_{p_{\mathsf{Vesta}}},$$

with bsk the *binding signing key*. The transaction declares its net flow in a signed cleartext field $v^{\mathsf{Orchard}}_{\mathsf{balance}}$, with the convention that a positive value is net value flowing *out* of the pool (paying fees or transparent outputs, say). The network subtracts the declared part:

$$\mathsf{bvk} := \sum_i \mathsf{cv}^{\mathsf{net},i} - [\, v^{\mathsf{Orchard}}_{\mathsf{balance}} \,]\, V^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

Proposition 7.4 (Balance equals key knowledge)**.** *If and only if the transaction balances — the summed value component equals the declared* $v^{\mathsf{Orchard}}_{\text{balance}}$ *— does the* V^{Orchard}*-component cancel, leaving* $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$*: a public key whose secret* bsk *the transaction author knows.*

The idea is already visible in the displays: by the homomorphism the aggregate commitment splits into a V-component carrying the net value difference and an R-component carrying bsk; subtracting the declared flow leaves a pure R-multiple exactly when the books balance, and only the author — who chose the rcv_i — knows their sum. The full argument, including why an unbalanced author cannot know a discrete logarithm of bvk to base R^{Orchard}, is deferred: Proposition 7.4 is proved in §12.

Proving knowledge of bsk — via a *binding signature* under the key bvk — is therefore precisely proving that the accounts balance, with no amount revealed (the *Crypto Guide*, §"Binding signatures as a proof of knowledge of a discrete logarithm"). The signature scheme is RedPallas, constructed in §7.4, here instantiated on the base R^{Orchard} — the value-commitment randomness base, distinct from the spend-authorisation base $G^{\mathsf{SpendAuth}}$. A single signature over the whole transaction (concretely, over the ZIP-244 *sighash* of §7.3) additionally binds all its Actions together, so that no one can remove or transplant any of them.

From NU6.3 — the upgrade that opens the Ironwood pool (§1) — the construction runs once *per pool*: a v6 transaction (§9) carries independent $v^{\mathsf{Orchard}}_{\text{balance}}$ and $v^{\mathsf{Ironwood}}_{\text{balance}}$ fields, and each pool's Actions sum to their own bvk, checked by their own binding signature over the same bases $V^{\mathsf{Orchard}}, R^{\mathsf{Orchard}}$ (ZIP-229). The Ironwood component's conservation is thus this subsection verbatim, run over the Ironwood Actions' $\mathsf{cv}^{\mathsf{net}}$ values.

7.3 What the signatures sign: the ZIP-244 digest and its v6 form

The message both Orchard signatures sign — the *sighash* — is not the raw transaction but a non-malleable digest of it, which ZIP-244 assembles as a tree of personalised BLAKE2b-256 hashes, one node per transaction component. Its root is

$$\mathsf{sighash} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \mathsf{branchid},\ d_{\text{hdr}} \,\|\, d_{\text{trans}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}}\big),$$

where branchid is the 4-byte identifier of the active network upgrade — so a signature is valid only on the fork it was made for — and each $d_{\bullet}$ is a personalised digest of one component's data, an absent component hashing the empty string. A purely shielded Orchard payment leaves the transparent and Sapling branches empty, and then the sighash coincides with the transaction identifier (txid) itself, the hash by which the chain names the transaction (ZIP-244).

The Orchard branch commits to the entire bundle,

$$d_{\text{Orch}} := \text{BLAKE2b-256}\big(\texttt{ZTxIdOrchardHash},\ d_{\text{C}} \,\|\, d_{\text{M}} \,\|\, d_{\text{N}} \,\|\, \mathsf{flagsOrchard} \,\|\, \text{LE}_{64}(v^{\mathsf{Orchard}}_{\text{balance}}) \,\|\, \mathsf{anchor}\big),$$

through three digests that partition every Action's fields: the *compact* d_{C} over nf, cmx, epk and the first 52 bytes $C_{\text{enc}}[0{:}52]$ of the note ciphertext (§5); the *memo* d_{M} over the 512-byte $C_{\text{enc}}[52{:}564]$; and the *non-compact* d_{N} over $\mathsf{cv}^{\mathsf{net}}$, the re-randomised validating key rk (constructed in §7.4), the tail $C_{\text{enc}}[564{:}]$ and C_{out}. Beside them sit the bundle-shared fields: the one-byte flag field flagsOrchard (its bit assignments are laid out in §10; the v6 renaming in §9), the declared balance, and the anchor of §6.

Folding in every Action's nf, cmx, $\mathsf{cv}^{\mathsf{net}}$, rk and ciphertexts beside the shared flagsOrchard, balance and anchor, the sighash pins the exact bundle: no Action can be transplanted into another transaction nor any field altered without breaking it (ZIP-244). The spend-authorisation signature of §7.4 under each rk and the binding signature under bvk both sign this single value.

The v6 form. From NU6.3 the tree gains an Ironwood branch (Figure 4):

$$\text{sighash} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \text{branchid},\ d_{\text{hdr}} \,\|\, d_{\text{trans}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}} \,\|\, d_{\text{Irw}}\big),$$

with five new Ironwood personalisations — `ZTxIdIronwd_H_v6` for the branch, `ZTxIdIrnActCH_v6`, `ZTxIdIrnActMH_v6`, `ZTxIdIrnActNH_v6` for its compact, memo, and non-compact digests, and `ZTxAuthIrnwdH_v6` on the authorizing-data side — and four revised v6 personalisations for the Sapling and Orchard branches (`ZTxIdSSpendNH_v6`, `ZTxAuthSapliH_v6`, `ZTxIdOrchardH_v6`, `ZTxAuthOrchaH_v6`); an absent component hashes the empty string under its personalisation (ZIP-229).

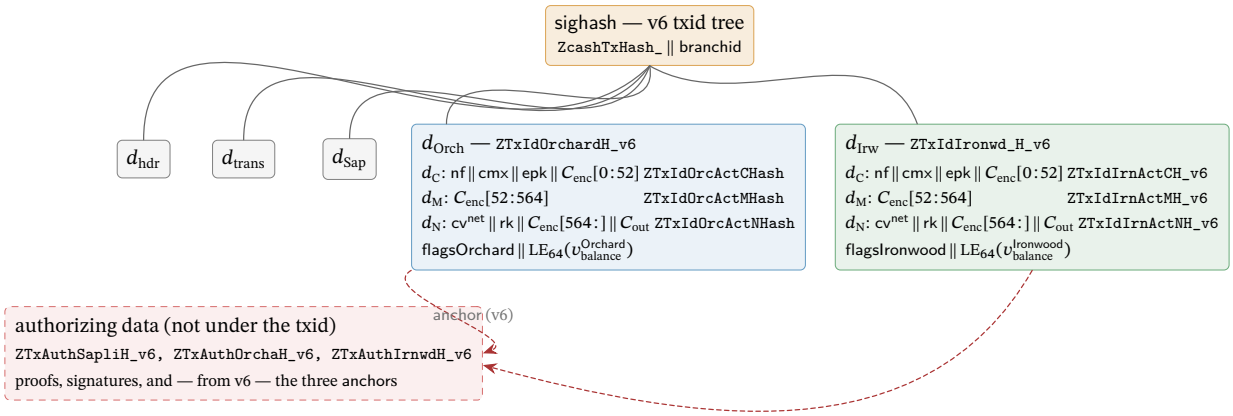

Figure 4: The v6 sighash as a tree of personalised BLAKE2b-256 nodes (ZIP-244; ZIP-229), with the Orchard and Ironwood branches expanded. Each node is one BLAKE2b-256 call whose personalisation (`typewriter`, right-aligned) separates its domain; the three Action digests $d_{\text{C}}, d_{\text{M}}, d_{\text{N}}$ run over the concatenation of the named fields across all of the branch's Actions. An absent component hashes the empty string under its personalisation. The Ironwood branch reuses the Orchard partition byte-for-byte under its own personalisations — flagsIronwood being the pool's own flag byte, the v6 renaming of flagsOrchard — closing the format split with §9. From v6 the anchors leave the txid branches for the authorizing-data digest (dashed), so a transaction can be re-anchored without changing its txid.

This digest tree is deployed. The released crate `orchard` 0.15.0 (the `librustzcash` pin) defines the seven Orchard and Ironwood `_v6` personalisations and selects them per pool and transaction version (`bundle/commitments.rs`, enum `BundleCommitmentFormat`), and `librustzcash` at `e30517e4` assembles the v6 digests in production: `digest_orchard` and `digest_ironwood` in `zcash_primitives transaction/txid.rs` call `Bundle::commitment`, the file itself defines only the two Sapling personalisations `ZTxIdSSpendNH_v6` and `ZTxAuthSapliH_v6` (lines 44 and 54), and `transaction/tests.rs` re-derives the empty-component digests as test vectors. The `bef8a27` dev tree predates this work: there `bundle/commitments.rs` (`:7--11`) defines only the five non-versioned personalisations (`ZTxIdOrchardHash`, `ZTxIdOrcActCHash`, `ZTxIdOrcActMHash`, `ZTxIdOrcActNHash`, `ZTxAuthOrchaHash`), with no per-pool or per-version dispatch.

One structural change rides along with v6: it moves the Sapling, Orchard, and Ironwood *anchors* from effecting data (under the txid) to authorizing data (under the auth digest, committed only when the component spends). A transaction can then be re-anchored — its membership proofs rebased onto a later root — without changing its txid, a design ZIP-229 credits to Penumbra.

7.4 Spend authorisation: RedPallas re-randomisation

One capability remains unproven: that the spender is *authorised* to spend the note — that they hold ask (§3) — established without linking this spend to any other spend by the same key. RedPallas,

a re-randomisable Schnorr signature, provides exactly this. It is the Schnorr template of the *Crypto Guide*, §"RedDSA: re-randomisable Schnorr for Orchard", instantiated on Pallas, with hash H^* — BLAKE2b-512 reduced mod p_{Vesta}.

The novel ingredient is *key re-randomisation*: the spender draws a randomiser $\alpha \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ uniformly at random, fresh for every Action — RedPallas's GenRandom hashes 80 uniform bytes with H^*, making α statistically uniform on $\mathbb{F}_{p_{\mathsf{Vesta}}}$ (reduction bias $O(2^{254}/2^{640}) = O(2^{-386})$) — and sets

$$\mathsf{rsk} := \mathsf{ask} + \alpha \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \qquad \mathsf{rk} := \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\, G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

The spender publishes the *randomised* validating key rk, not the real ak, and signs the sighash with rsk.

Unlinkability is retained alongside authority. Because α is fresh per spend, the key rk is uniformly random and unlinkable across spends (the *Crypto Guide*, §"Key re-randomisation and unlinkability"), yet remains a valid key for the underlying authority. The circuit enforces that rk is a re-randomisation of the witnessed ak (the key re-randomisation check, §8) and that this ak is the key bound into the note's address (the CommitIvk check, §8), so a valid Action proves the spender knows ask — without ever revealing ak.

> **Proposition 7.5** (Unforgeability under re-randomisation)**.** *RedPallas is existentially unforgeable under chosen-message attack (EUF-CMA; the* Crypto Guide*, §"Syntax and security goal") in the random oracle model under discrete-logarithm hardness on Pallas, and remains so under key re-randomisation: signatures under re-randomised keys* $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$*, for randomisers* α *known to the adversary, give no advantage in forging under* ak *or any of its re-randomisations.*

Proposition 7.5 is proved in §12.

The spend's contributions are now complete. Alongside the delivery fields $(\mathsf{epk}, C_{\mathrm{enc}}, C_{\mathrm{out}})$ of §5, an Action publishes the nullifier nf, the net value commitment $\mathsf{cv}^{\mathsf{net}}$, the randomised validating key rk with its spend-authorisation signature, and the new note's cmx (§4); the transaction adds one binding signature per pool. What remains is to assemble these pieces into the Action and its zero-knowledge statement — the work of §8.

8 The Action assembled: walkthrough, verification, and the statement

The pieces are on the table: an address (§3), a note and its commitment (§4), the delivery ciphertexts (§5), the tree (§6), and the spend's nullifier, value commitment, sighash, and signatures (§7). This section assembles them: one payment end to end, the network's side of it, and the formal statement carrying both. The unit of shielded transfer is the *Action*: one Action simultaneously spends one old note and creates one new note — a deliberately fixed shape that conceals whether a given Action is "really" a spend, an output, or both.

8.1 One Action, one payment: the walkthrough

Consider the simplest possible payment: Alice pays Bob 5 ZEC ($v = 5 \cdot 10^8$ zatoshi, at 10^8 zatoshi per ZEC) from a note worth exactly 5 ZEC. This payment is one Action. (The names A1–A10 are the checks of Definition 8.1 below.)

1. **Bob hands Alice an address.** Bob derives a diversified address $(d, \mathsf{pk_d})$ from his ivk (Construction 3.5, §3) and gives it to Alice. It reveals nothing and is unlinkable to his other addresses.

2. **Alice builds the Action.** Alice owns an old note $\mathbf{n}_{\mathrm{old}}$ worth 5 ZEC, whose commitment $\mathsf{cmx}_{\mathrm{old}}$ is a leaf of the tree. She computes its nullifier $\mathsf{nf}_{\mathrm{old}}$ (Definition 7.1), which its publication will retire;

constructs Bob's new note $\mathbf{n}_{\text{new}} := (\text{addr}_{\text{Bob}}, 5 \cdot 10^8, \rho_{\text{new}}, \psi_{\text{new}}, \text{rcm}_{\text{new}})$ with $\rho_{\text{new}} := \text{nf}_{\text{old}}$, and its commitment cmx_{new} (Construction 4.2); forms the net value commitment cv^{net} with $v_{\text{old}} - v_{\text{new}} = 5 \cdot 10^8 - 5 \cdot 10^8 = 0$ (Definition 7.3); selects a randomiser α and forms rk (§7.4); and reads off a recent anchor and her note's Merkle path (Construction 6.2, §6).

3. **Alice proves the Action statement.** She runs the Halo 2 prover on the Action relation, with public inputs $(\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx}_{\text{new}})$ and her note, keys, path, and randomness as private witness. The proof attests, in one shot and revealing nothing: the old note is in the tree (A3), its nullifier follows correctly (A5), she is authorised to spend it (A6, A7), the spent and created notes are well-formed (A1, A2), and the value commitment is consistent (A4).

4. **Alice assembles and signs the transaction.** She encrypts $\mathbf{n}_{\text{new}}$ to Bob (§5), attaches a *spend-authorisation signature* (SpendAuthSig) under rk (§7.4), and signs the bundle with the binding signature under bvk (§7.2), which proves balance. Both signatures sign the ZIP-244 sighash (§7.3), welding the Action into this transaction.

5. **The network verifies.** Consensus checks that it recognises the anchor, that nf_{old} is not already in the nullifier set (no double-spend), that the Halo 2 proof verifies, and that the SpendAuthSig under rk and the binding signature under the recomputed bvk verify (authority; balance). If all pass, it adds nf_{old} to the nullifier set and appends cmx_{new} as a new tree leaf.

6. **Bob receives.** Scanning with his ivk, Bob trial-decrypts the ciphertext (§5), recovers $\mathbf{n}_{\text{new}}$, and now owns a 5-ZEC note he can later spend by repeating the entire procedure.

That sequence constitutes the entire protocol; the formal Action statement of §8.3 is precisely the list of in-circuit checks step 3 invokes. Equal values are the special case: a more valuable old note would return the difference to Alice as change in a further note, subject from NU6.3 to the routing rule constructed in §10.

After the payment, the chain has gained one nullifier and one commitment; an observer learns that *an* Orchard payment occurred and nothing else — not sender, recipient, or amount. The claim rests on the proof's zero knowledge and the composition of every piece of the journey; §12 states and proves it.

8.2 What a node verifies, without ever observing the note

Nothing in the walkthrough handed the network a note. A validating node holds no viewing key, cannot trial-decrypt, and so never learns the sender, recipient, or amount of any Action; all it can confirm is that the hidden data is internally consistent, and for that it relies entirely on the proof, never on the note.

What the node receives is an Orchard-protocol *bundle*, the unit into which the transaction format groups one pool's data: the per-Action public fields; *one* anchor shared by every Action in the bundle; a *single* aggregate Halo 2 proof covering all of the bundle's Actions; and one SpendAuthSig per Action (ZIP-225; crate `zebra-chain`, `src/orchard/shielded_data.rs`). The declared net flow $v_{\text{balance}}^{\text{Orchard}}$ and the binding signature (§7.2) appear once per bundle, and the flags enableSpends, enableOutputs, and — from NU6.3 — enableCrossAddress (bit 2 of the flags byte flagsOrchard of §7.3, reserved as 0 in v5 transactions before NU6.3) are likewise bundle-level fields (ZIP-225 reserves the bit; ZIP-229 assigns it).

Using only the public fields $(\text{anchor}, \text{cv}^{\text{net}}, \text{nf}_{\text{old}}, \text{rk}, \text{cmx})$ of each Action, the node checks:

- the bundle's *Halo 2 proof* of the Action statement (§8.3) — this forces each spent note of nonzero witnessed value to be a genuine tree leaf (a zero-value *dummy* spend, the shape by which an Action pads with no real input, skips the membership check by A3's gate, by design), the nullifiers and new commitments to follow correctly, and the keys to align;
- the *anchor* is a Merkle root the node has seen on its accepted main chain (§6.4);

- the nullifier $\mathsf{nf}_{\mathsf{old}}$ is *not already* in the nullifier set — the one check that cannot sit inside a single proof, because it concerns the global history (this prevents a double-spend);
- the *spend-authorisation signature* verifies under rk and the *binding signature* under the recomputed bvk (§§7.2–7.4), settling authority and balance.

If every Action passes, the node performs the two state changes of step 5 — recording each $\mathsf{nf}_{\mathsf{old}}$, appending each $\mathsf{cmx}_{\mathsf{new}}$ — which permit the next spender to prove membership and make a second spend of the same note fail. Thus the SNARK together with the two signatures enforces a payment's validity, never the inspection of the note: the node confirms that the accounts balance and that no one forged anything *while learning nothing about the note itself*. This is the precise sense in which the knowledge soundness of the Action SNARK performs all the work — the network trusts the proof exactly because a real witness extractably backs a valid one (the extraction argument of §12).

8.3 The Action statement, formally

The relation itself remains. The proof the node verifies is a Halo 2 proof (the *Halo 2 Guide*, §"The Halo 2 proof system"); how the relation becomes a circuit is §11's subject, and what its soundness yields is §12's. Every check is typed over primitives the *Crypto Guide* constructs — commitments (§"Commitment schemes"), PRFs (§"Pseudorandom functions and permutations"), Schnorr keys (§"Digital signatures") — as instantiated on Pallas in §§2–7, and closes with the requirement it discharges.

Definition 8.1 (Orchard Action statement, Ironwood circuit (NU6.3))**.** The Action SNARK proves, with public inputs

$$(\mathsf{anchor},\ \mathsf{cv}^{\mathsf{net}},\ \mathsf{nf}_{\mathsf{old}},\ \mathsf{rk},\ \mathsf{cmx},\ \mathsf{enableSpends},\ \mathsf{enableOutputs},\ \mathsf{disableCrossAddress})$$

— eight named inputs; the two Pallas points enter by affine coordinates, so the instance vector has ten elements — and with the two notes' fields, the keys, the Merkle path, the randomiser α, and the value-commitment trapdoor rcv as private witness, the conjunction:

[A1] Old-note commitment integrity — an equation between Pallas points, over the Sinsemilla commitment of Construction 4.2:

$$\mathsf{cm}_{\mathsf{old}} = \mathsf{NoteCommit}_{\mathsf{rcm}_{\mathsf{old}}}\big(\mathsf{g}_{\mathsf{d}}^{\star}{}_{\mathsf{old}} \,\|\, \mathsf{pk}_{\mathsf{d}}^{\star}{}_{\mathsf{old}} \,\|\, \mathrm{LE}_{64}(v_{\mathsf{old}}) \,\|\, \mathrm{LE}_{255}(\rho_{\mathsf{old}}) \,\|\, \mathrm{LE}_{255}(\psi_{\mathsf{old}})\big)$$

— or $\mathsf{NoteCommit}$ returns $\perp$, the specification's escape hatch for Sinsemilla's incomplete-addition exceptions (Remark 8.2). The spent object really is a note (R1's representation).

[A2] New-note commitment integrity, with $\rho_{\mathsf{new}} := \mathsf{nf}_{\mathsf{old}}$: over the same commitment and the coordinate extraction of §2.1,

$$\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}_{\mathsf{new}}}(\ldots)\big) \in \{\mathsf{cmx}, \perp\} \qquad (\text{with } \mathsf{Extract}(\perp) := \perp).$$

The created note is well-formed (R1's representation again), and its uniqueness seed chains to the spent note's nullifier — the Faerie resistance of §7.1.

[A3] Membership, gated by v_{old} — a Merkle recomputation over the layer-tagged hash of Definition 6.1: recomputing the root from $\mathsf{Extract}(\mathsf{cm}_{\mathsf{old}})$ up the witness path of length 32 gives root, and $v_{\mathsf{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$ in $\mathbb{F}_{p_{\mathsf{Pallas}}}$. The gate permits a zero-value dummy spend to skip the membership check — how an Action pads with no real input. The spent note is a real leaf of the tree (R2).

[A4] Net-value commitment integrity — the Pedersen commitment of Definition 7.3: with witnessed (magnitude, sign),

$$v_{\text{old}} - v_{\text{new}} = \text{magnitude} \cdot \text{sign}, \qquad \mathsf{cv}^{\mathsf{net}} = [\, v_{\text{old}} - v_{\text{new}} \,]\, V^{\mathsf{Orchard}} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}},$$

with $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$ and sign $\in \{-1, +1\}$. The amounts are consistent and in range: value is conserved (R4).

[A5] Nullifier integrity — the PRF-plus-group derivation of Definition 7.1:

$$\mathsf{nf}_{\text{old}} = \mathsf{Extract}\big([\, F_{\mathsf{nk}}(\rho_{\text{old}}) + \psi_{\text{old}} \,]\, G^{\mathsf{nf}} + \mathsf{cm}_{\text{old}}\big).$$

The published nullifier is indeed this note's; with the chain's nullifier set, this prevents double-spending (R3).

[A6] Spend authority — a Schnorr-key re-randomisation (§7.4): $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$. The published validating key belongs to the note owner's spend authority.

[A7] Address integrity — the short commitment $\mathsf{Commit}^{\mathsf{ivk}}$ and the scalar multiplication of Construction 3.4: $\mathsf{ivk} = \bot$, or

$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}\big(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}\big) \quad \text{and} \quad \mathsf{pk_{d_{old}}} = [\mathsf{ivk}]\, \mathsf{g_{d_{old}}}.$$

The keys, the address, and the nullifier key all belong to one identity; with A6, the spender is authorised (R1's possession clause).

[A8] / [A9] Spend / output gating — field products against the bundle flags of §8.2: $v_{\text{old}} \cdot (1 - \mathsf{enableSpends}) = 0$ and $v_{\text{new}} \cdot (1 - \mathsf{enableOutputs}) = 0$. The transaction builder sets these bundle-level flags and consensus rules constrain them: coinbase transactions (those minting each block's subsidy, §4.3) must set $\mathsf{enableSpends} = 0$ (ZIP-229). Spending Ironwood notes is otherwise permitted — the $\mathsf{enableSpends} = 0$ requirement is coinbase-scoped, not a pool-wide prohibition (protocol specification § 7.1, transaction consensus rules).

[A10] Cross-address gating — when $\mathsf{disableCrossAddress} \neq 0$, the created note pays the spent note's own receiver: $\mathsf{g_{d_{new}}} = \mathsf{g_{d_{old}}}$ and $\mathsf{pk_{d_{new}}} = \mathsf{pk_{d_{old}}}$ as full curve points, enforced by four product constraints of the A3 shape, one per affine coordinate. The flag is a verifier-supplied boolean public input — the tenth, at instance index 9; its semantics, its instance and wire encodings, and the consensus requirement that every post-NU6.3 Orchard-pool Action set it are all constructed in §10.

Remark 8.2 (The $\bot$-weakening)**.** Checks A1, A2, and A7 take the specification's $\bot$-weakened forms because the circuit cannot exclude Sinsemilla's incomplete-addition exceptional cases: the relation proved is the disjunction, not the plain conjunction. The weakening costs only a negligible soundness term — producing a $\bot$ case is itself a non-trivial discrete-logarithm relation among the GroupHash generators (the closing step of the proof of Theorem 2.2, given in §12), so no efficient prover exhibits one.

Each of A1–A7 is one entry from the four-requirement map of §1: A1–A2 represent notes, A3 proves membership, A5 (with the chain's nullifier set) prevents double-spending, A4 conserves value, and A6–A7 authorise the spender; A8–A10 stand outside the map as consensus-level gating constraints. This is the relation $\mathcal{R}_{\mathsf{Orchard}}$, and every Action is an instance of it. Halo 2 produces one aggregate proof *per bundle*, covering all of the bundle's Actions, while each Action carries its own SpendAuthSig. Figure 5 draws the statement in one place: every public field and every witness component, the checks each feeds, and, per check, the object it constrains and the section that constructed it. How the Action circuit arithmetises this relation is §11's subject; how its knowledge soundness yields the protocol's security properties is §12's.

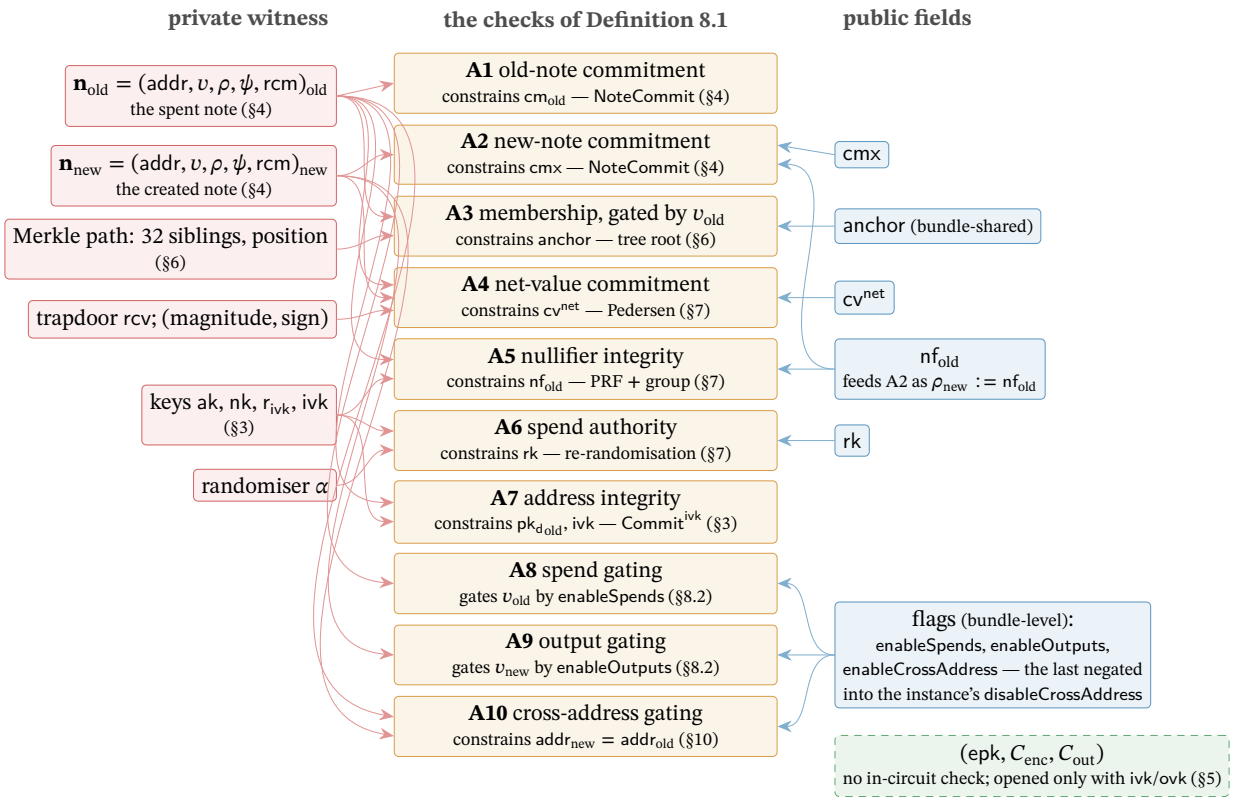

Figure 5: The Action statement as the volume's hub. Centre: the ten checks of Definition 8.1, each annotated with the object it constrains and the section that constructed it. Left: the private witness; right: the Action's public fields, with the bundle-level flags below. Arrows run from each input to the checks it feeds; the edge $\rho_{\text{new}} := \text{nf}_{\text{old}}$ is the Faerie chain of §7.1, tying the created note's uniqueness seed to the spent note's nullifier. The delivery ciphertexts (dashed) feed no check: the proof never touches them, and only the holder of ivk or ovk can open them (§5).

9 Two pools, one protocol: doors and the v6 transaction

Every stage so far has moved value *inside* a shielded pool: note to note, owner to owner, with the Action of §8 as the unit of transfer. Value also crosses the pool boundary — entering from the transparent ledger, arriving as fresh issuance, or migrating between shielded pools — and each crossing is public in amount, by design: what a pool hides is who holds which note, never how much the pool as a whole contains. This section formalises the three doors through which value enters, the ledger that meters every crossing, and the transaction format that carries a two-pool bundle; it closes with the piece of the split most easily missed — that the two pools share one circuit and one verifying key.

The split itself is already in hand. The NU6.3 network upgrade (ZIP-258, Draft) seals the value pool that has carried Orchard payments since NU5 and starts a second pool, *Ironwood*, running the same cryptography. ZIP-229 (Draft) fixed the vocabulary and §1 adopted it: one *Orchard protocol* — the shared design, its Action circuit as modified by ZIP-2006 and its note encryption as modified by ZIP-2005 — and two value pools of that one protocol, the sealed *Orchard pool* and the current *Ironwood pool*, each with its own note commitment tree, anchor, and chain value pool balance (§6.5). Nothing in the preceding sections is discarded; what changes is which pool new value may enter — this section's subject — and how an Ironwood note derives its trapdoor, already constructed as Definition 4.3 (§4).

The volume's deployed-behaviour citations are pinned to a fixed baseline, stated here once: the released crate `orchard` 0.15.0 (the version `librustzcash` pins; the Ironwood note-format, note-encryption, and v6-digest work ships in it), the `orchard` git tree at `bef8a27` (a 0.14.0 development tree predating the release, cited for the circuit and cross-address work it carries), `librustzcash` at commit `e30517e4`, and `protocol.tex` and the ZIPs at `zips` commit `69610984`. The later deployment and migration status of ZIPs 258 and 318 is rechecked against `zips` commit `dd1667ff` (2026-09-01). Forward-looking claims carry one of three epistemic bins: *specified*, *designed but unspecified*, or *open problem*.

9.1 The doors: shielding, coinbase, migration

Value enters a shielded pool through three doors, each public in amount.

The first door is the *shielding* transaction, and its instrument is the sign of the balancing value: by the convention of §7.2, a positive $v_{\text{balance}}^{\text{Orchard}}$ is net value leaving the pool, so a shielding transaction is one whose transparent inputs fund a *negative* balancing value. Its Actions spend *dummy* notes — zero-value placeholders, $v_{\text{old}} = 0$, for which the Action statement gates off the membership check (check A3, §8) — and create real ones, and the binding signature of §7.2 proves the books balance. An observer learns the entering amount — the balancing field is a cleartext transaction field — but not the receiving note or address. The deployed construction path is function `propose_shielding` (`zcash_client_backend`, `data_api/wallet.rs` and `input_selection.rs`): it sweeps the given source addresses' UTXOs (§6), at zero confirmations under the default policy, and wallets auto-shield transparent receipts to *internal* keys — address material the wallet derives for itself and never publishes. From NU6.3 this door opens into the Ironwood pool alone: a shielding transaction funds a negative $v_{\text{balance}}^{\text{Ironwood}}$, the Orchard entry being sealed (§10).

The second door is *shielded coinbase* (ZIP-213). Since the Heartwood network upgrade, new issuance may enter a shielded pool directly — "[c]oinbase transactions MAY contain Sapling outputs" — extended at NU5 to the Orchard pool and re-scoped at NU6.3: from then coinbase may carry "Sapling-pool and/or Ironwood-pool outputs", and "it is only possible to mine to an Orchard address by sending the funds to the Ironwood pool" (ZIP-213). Two riders keep this auditable and consistent. First, coinbase *maturity* — the rule that a block's newly minted outputs may not be spent until a fixed depth has passed — is amended to bind transparent outputs only; the amended rule is the one the *Consensus Guide* reads through its theorem in §"Reorg rails, maturity, and the finality floor". Second, every shielded coinbase output "MUST have valid note commitments" when recovered under the *all-zero* outgoing viewing key ovk (§3): coinbase shielding is publicly decryptable by construction,

so anyone can verify what issuance entered which pool, while the notes spend exactly like any others afterwards. An Ironwood coinbase output also carries lead byte `0x03`, the rule of Definition 4.4 (§4.3).

The third door is *migration*: value moving from one shielded pool into another without touching the transparent ledger. Between this volume's two pools, migration passes through the one-way turnstile — NU6.3's rules confining the sealed Orchard pool to outward flow — constructed in §10. Here it is enough that migration, like the other two doors, is public in amount, because every door writes to the ledger we now examine.

9.2 Pool balances: ZIP-209 and its Ironwood analogue

Every entry and exit is metered by ZIP-209's pool balances. For each shielded pool, the *chain value pool balance* is the negation of the sum of that pool's balancing fields over the whole chain — for the earliest shielded pool, Sprout, which declared entry and exit in a pair of cleartext fields, it is the sum of `vpub_old` − `vpub_new` — and a block is invalid if it would drive any pool balance negative — shielded, transparent, or the deferred fund (ZIP-2001's *lockbox*, a chain value pool accruing a portion of each block subsidy for later disbursement) — or total supply past the protocol's fixed cap MAX_MONEY (ZIP-209; ZIP-2001). Entry, transfer, and exit therefore reconcile publicly at pool granularity while individual notes stay hidden.

From NU6.3 the ledger gains one row. The Ironwood pool keeps its own chain value pool balance (§6.5), the negation of the summed $v^{\mathsf{Ironwood}}_{\mathsf{balance}}$ fields of §7.2, under the same non-negativity rule. The doors of §9.1 write to that row — a shielding transaction or a shielded coinbase output raises the Ironwood balance, value leaving through a positive balancing field lowers it — while the sealed Orchard pool's balance can only fall: the accounting shape of the turnstile, whose rules §10 states.

9.3 The v6 transaction and the Ironwood component

The upgrade has a concrete identity. NU6.3 carries consensus branch identifier `0x37A5165B`; ZIP-258 now records both networks' activation heights and sets minimum network protocol version 170160 on both. The deployed validator fixes the same values in `zebra-chain/src/parameters/constants.rs`, with `librustzcash` hardcoding the same values in `zcash_protocol/src/consensus.rs`. ZIP-258 remains Draft, but these deployment parameters are no longer to-be-determined. From activation onward, the ZIP expects Zebra to be the network's sole full-validator implementation.

A two-pool bundle needs a carrier, and the current carrier is the *v6 format* (ZIP-229): the v5 format's Orchard component (ZIP-225) plus a separate Ironwood component holding that pool's bundle (flagsIronwood, valueBalanceIronwood, anchorIronwood, proofsIronwood, vSpendAuthSigsIronwood, bindingSigIronwood). Concretely, a v6 transaction (`V6_TX_VERSION = 6`, version group `0xD884B698`; `librustzcash` `zcash_protocol/src/constants.rs`) is the v5 format — with enableSpendsOrchard/enableOutputsOrchard renamed to enableSpends/enableOutputs, and bit 2 of flagsOrchard now carrying enableCrossAddress, the flag whose semantics §10 constructs — plus an Ironwood component that reuses the `OrchardAction` encoding byte-for-byte (820 bytes per Action; ZIP-229), laid out field by field in Table 1. Transaction versions v4, v5, and v6 all remain valid from NU6.3 onward, and the Orchard-pool sealing rules bind regardless of version (§10).

Two pointers close the format. At the note level, ZIP-229 states the situation exactly: every Ironwood-pool output note uses the quantum-recoverable note plaintext format — lead byte `0x03` with the hashed trapdoor of Definition 4.3 (§4) — no Orchard-pool output note does, and this is "the only note-level distinction between the two pools". At the digest level, the v6 sighash tree gains an Ironwood branch mirroring the Orchard one, already constructed alongside the spend's signatures (Figure 4, §7.3). Everything NU6.3 changes is collected, one row per change with its owning section, in Table 2 (§10).

Field	Size (bytes)	Content
nActionsIronwood	`compactSize`	Action count n
vActionsIronwood	$820 \cdot n$	`OrchardAction` encoding
flagsIronwood	1	same bit layout as flagsOrchard
valueBalanceIronwood	8	net flow $v_{\text{balance}}^{\text{Ironwood}}$
anchorIronwood	32	a root of the *Ironwood* tree
sizeProofsIronwood	`compactSize`	length of proofsIronwood
proofsIronwood	$2720 + 2272 \cdot n$	aggregate Halo 2 proof
vSpendAuthSigsIronwood	$64 \cdot n$	one SpendAuthSig per Action
bindingSigIronwood	64	the pool's binding signature

Table 1: The Ironwood component of a v6 transaction (ZIP-229). Sizes marked `compactSize` use Bitcoin's variable-length integer encoding. The fields after the count are present iff $n > 0$. The proof size formula is the same $2720 + 2272n$ as the v5 Orchard component's proofsOrchard (ZIP-225).

9.4 One circuit, one verifying key

For all this new state, nothing new is proved. Both pools share the post-NU6.3 Action circuit and its proving and verifying keys; the pools are distinguished by their trees, nullifier sets, balances, and component position — not by separate circuits. The released `orchard` 0.15.0 encodes the pairing precisely: a `BundleVersion` pairs a `ValuePool` (`Orchard` or `Ironwood`) with a `ProtocolVersion` (`InsecureV1`, `V2`, or `V3`); `circuit_version()` maps `V3` to `OrchardCircuitVersion::PostNu6_3` for *both* pools; `note_version()` maps `Orchard` to `V2` and `Ironwood` to `V3` — the lead-byte split of §4 — and `permits_cross_address_transfers()` is false exactly for the pair (`V3`, `Orchard`) (`src/lib.rs`, `src/bundle.rs`). The in-circuit gate is `supports_cross_address_restriction()` on `OrchardCircuitVersion`, true only for `PostNu6_3` (`src/circuit.rs`); the `bef8a27` development tree's `BundleFormat` enum and `Ironwood` circuit-version name were reworked into these before release.

Remark 9.1 (The three Action-circuit versions)**.** The deployed implementation distinguishes three Action-circuit versions: the insecure NU5 original, the NU6.2 correction (ZIP-257, Final), and the NU6.3 Ironwood circuit — and only the last constrains the cross-address flag (`OrchardCircuitVersion::supports_cross_address_restriction`, released crate `orchard` 0.15.0, `src/circuit.rs`). The legacy NU5/NU6 statement is the relation A1–A9 alone (§8), over the nine-element public-input tuple ending at enableOutputs. Consensus selects the verifying key by branch (ZIP-258), so from NU6.3 every Action — in either pool, whatever the transaction version — verifies against the Ironwood key. How the stack came to have three versions — what the original got wrong and what the correction changed — is §13's story; this remark is the operational list that story cites.

One door remains open. The shielding and coinbase doors lead into the Ironwood pool; leaving the sealed Orchard pool — the migration the first subsection deferred — passes through a turnstile with rules of its own: the seal, the cross-address restriction the verifying-key selection just made binding, and the one-way flow the pool balances record. Constructing that turnstile is the next section's work (§10).

10 The seal and the turnstile: leaving the Orchard pool

Every door of §9 opens inward, and from NU6.3 all of them open into the Ironwood pool. This section constructs the outward passage that remains: how value held in the sealed Orchard pool leaves it. Consensus arranges the exit as two instruments — a *seal*, three rules that stop value entering the pool or circulating inside it, and a *turnstile*, the one-way, publicly metered gate through which the sealed pool drains. We take them in order: the three rules, the in-circuit restriction that is the deepest of

them, what spending under that restriction looks like, the turnstile and the migrating transaction, and the wallet posture built on top — closing with the one-place table of everything NU6.3 changes (Table 2).

10.1 The seal: three rules

From NU6.3 activation, three consensus rules seal the Orchard pool to spend-only, and all three bind every transaction *regardless of its version* (ZIP-258):

1. **No issuance.** A coinbase transaction must carry an *empty* Orchard component: the shielded-coinbase door of §9.1 no longer opens into the Orchard pool.
2. **No circulation.** Every Orchard-pool Action must have cross-address transfers disabled, $\mathsf{enableCrossAddress} = 0$ (§10.2). The rule is enforced by the Action-circuit verifying key, which consensus selects by branch (Remark 9.1, §9.4), so a v5 transaction cannot bypass it — ZIP-229 states this version-independence explicitly.
3. **No entry.** Per transaction, $v_{\text{balance}}^{\text{Orchard}} \geq 0$: net value may flow out of the pool, never in (§10.4).

The deepest of the three is the second, because it lives in the circuit: from NU6.3, value inside the Orchard pool can no longer move between addresses — an Orchard-pool spend can pay only its own receiver. The motivation is the recoverability guarantee whose analysis §14.5 owns: legacy lead-byte-`0x02` notes (§4.3) are unrecoverable, so consensus stops their circulation and funnels the pool's value out through the turnstile (§10.4), while still letting owners spend what they hold.

10.2 The cross-address restriction (A10)

Definition 8.1 (§8) listed check A10 and deferred its semantics to this section. We give the constraint in full, then its two encodings — in the proof's instance vector and on the wire — and the backward compatibility both preserve.

Definition 10.1 (The A10 constraint in full)**.** The tenth public input of Definition 8.1, $\mathsf{disableCrossAddress}$, sits at instance index 9 — after anchor, the two affine coordinates of $\mathsf{cv}^{\mathsf{net}}$, $\mathsf{nf}_{\mathsf{old}}$, the two coordinates of rk, cmx, enableSpends, and enableOutputs. Writing $\delta := \mathsf{disableCrossAddress}$, and $x(\cdot), y(\cdot)$ for a Pallas point's affine coordinates (§2.1), constraint A10 (cross-address gating) is the conjunction

$$\delta \cdot \big(x(\mathsf{g}_{\mathsf{d}_{\mathsf{old}}}) - x(\mathsf{g}_{\mathsf{d}_{\mathsf{new}}})\big) = 0, \qquad \delta \cdot \big(y(\mathsf{g}_{\mathsf{d}_{\mathsf{old}}}) - y(\mathsf{g}_{\mathsf{d}_{\mathsf{new}}})\big) = 0,$$
$$\delta \cdot \big(x(\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}}) - x(\mathsf{pk}_{\mathsf{d}_{\mathsf{new}}})\big) = 0, \qquad \delta \cdot \big(y(\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}}) - y(\mathsf{pk}_{\mathsf{d}_{\mathsf{new}}})\big) = 0$$

in $\mathbb{F}_{p_{\mathsf{Pallas}}}$: four coordinate equalities which, whenever $\delta \neq 0$, force $\mathsf{g}_{\mathsf{d}_{\mathsf{old}}} = \mathsf{g}_{\mathsf{d}_{\mathsf{new}}}$ and $\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}} = \mathsf{pk}_{\mathsf{d}_{\mathsf{new}}}$ as full curve points — equality of receivers, not merely of x-coordinates. (Crate `orchard` at `bef8a27`, `src/circuit.rs`: instance offsets `ANCHOR` through `DISABLE_CROSS_ADDRESS`; rows assigned in `synthesize_cross_address_checks`.)

The constraint shape is deliberately unoriginal: it is the product form check A3 already uses — $v_{\mathsf{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$ — reused on four extra rows of the same custom gate (the rows just cited from Definition 10.1's implementation). The addition also costs history nothing: the historical nine-element instance encoding is *commitment-identical* when $\delta = 0$, so pre-NU6.3 proofs and verifying keys are untouched by A10 (crate `orchard` at `bef8a27`, `src/circuit.rs`).

On the wire the flag travels in the opposite polarity: bit 2 of the component's flags byte (§9.3) carries $\mathsf{enableCrossAddress}$ — the *enabled* sense, negated into the instance's δ — with the bit assignments

spends `0b001`, outputs `0b010`, cross-address `0b100`. Deployed encoders carry both canonical flags values, `0b0000_0011` for an Orchard component and `0b0000_0111` for an Ironwood one (`librustzcash`, `pczt/src/orchard.rs`).

The polarity is backward-compatible by design. Bit 2 = 0 — the restricted state consensus requires of post-NU6.3 Orchard-pool spends — is exactly what v5 signers treating the bit as reserved-zero already produce, so existing hardware signers (ZIP-229 names the Keystone) keep signing v5 Orchard unshielding transactions — ones whose positive balancing value pays out of the pool (§9.1) — without a firmware update. The Ironwood component carries the same flag and *sets* it: cross-address transfers stay enabled there.

10.3 Spending under the restriction

An Action spends one note and creates one (§8.1); under the restriction the created note must pay the spent note's own receiver. An owner therefore pays anyone by *exiting the pool*: the real value leaves through a positive $v_{\text{balance}}^{\text{Orchard}}$ (§10.4), and the Action's output slot is filled with a *fabricated* zero-valued note to the spent note's own receiver — the fabricated outputs that keep the Orchard tree growing after the split (§6.5).

The fabrication must be thorough. The wallet fills the fabricated output's ciphertext C_{enc} (§5) with *random bytes*: a real encryption would let any ivk holder — including a quantum adversary who recovered ivk from a published address, the very adversary §14.2 confronts — decrypt it and link the Action's nullifier to the address (ZIP-326).

Between construction and signing, the pretence is dropped. In a PCZT — a *partially created Zcash transaction*, the format that carries an in-progress transaction between its constructor, prover, and signers — the fabricated output carries its recipient, value, and rseed explicitly, so a signer can classify it as a tolerable dummy rather than an opaque payment (ZIP-326; `librustzcash`, `pczt/src/orchard.rs`).

One gap remains open by construction. Paying into an Orchard-pool receiver post-NU6.3 requires a spend at that same receiver, hence the spending key, so consensus cannot fully prevent *self*-sends; ZIP-326's wallet rules close the gap by forbidding sends to external Orchard-pool receivers (external as against the internal address material a wallet derives for itself, §9.1).

A final oddity of provenance: the cross-address restriction is deployed and enforced by consensus, but its owning ZIP does not yet exist as text — ZIP-2006 ("Restricting Transfers into the Orchard Pool") is a nine-line Reserved stub, owner Daira-Emma Hopwood, created 2025-06-20, discussion `zcash/zips#1305`. In the bins of §9, *specified* (ZIP-229; ZIP-258) are the flag's encoding, its consensus rule, and the verifying-key selection by branch; *designed but unspecified* are the restriction's normative semantics — ZIP-326 phrases it as equality of "expanded receivers" and carries a `[TODO define]` in its own text — together with the enableCrossAddress → disableCrossAddress instance mapping and the dummy-spend treatment, reconstructable only from ZIP-229, ZIP-258, ZIP-326, and crate `orchard` at `bef8a27`, as Definition 10.1 does.

10.4 The turnstile and migration

The exit itself is older than the pool it now drains.

> **Definition 10.2** (Turnstile)**.** The *turnstile* is the mechanism by which value leaves the Orchard pool: a one-way, publicly metered gate, the same instrument Zcash used for the Sprout retirement (§13). Its consensus content consists of exactly two rules, both encountered already: the per-transaction non-negativity rule (the seal's third, §10.1) and the Ironwood pool-balance rule (§9.2), made precise below.

The first rule, once more and exactly: from NU6.3 activation, $v_{\text{balance}}^{\text{Orchard}} \geq 0$ holds per transaction, for *any* transaction version (ZIP-258). Value may exit the Orchard pool; it may never enter.

The second rule gives the destination pool its ledger row. Writing

$$\mathsf{IronwoodPoolBalance} := -\sum_{\mathrm{tx}} v_{\mathrm{balance}}^{\mathsf{Ironwood}}(\mathrm{tx}),$$

the chain maintains an Ironwood analogue of every pool-balance rule of §9.2 (the ZIP-209 extension): the balance is zero before NU6.3, may never go negative, and the sum of all pool balances is bounded by MAX_MONEY (ZIP-209; ZIP-258).

A migrating transaction, then, has a fixed shape: it spends Orchard-pool notes with $v_{\mathrm{balance}}^{\mathsf{Orchard}} > 0$ — a positive balancing field being net value flowing *out* of the pool (the sign convention of §7.2), into the transaction's transparent funds, from which fees and transparent outputs are also paid — and absorbs that value into an Ironwood component with $v_{\mathrm{balance}}^{\mathsf{Ironwood}} < 0$. Both balancing fields are cleartext: the turnstile *reveals the amounts crossing between pools in every transaction*, exactly as ZIP-229's privacy analysis states, and no per-transaction or aggregate cap on migration is specified anywhere.

The remaining Ironwood consensus rules are this volume's Orchard rules re-instantiated (ZIP-258). The anchor anchorIronwood must be an earlier main-chain block's final Ironwood treestate — the Ironwood tree as that block left it (§6.5) — and the Ironwood commitment tree has the same capacity 2^{32} as its Orchard twin (§6.5). Beyond the trees, the ZIP-221 *history tree* — the Merkle mountain range over the chain's history whose root every block header commits to — gains three Ironwood fields, the pool's earliest root, latest root, and transaction count, aggregated exactly like the Orchard trio (ZIP-221; the *FlyClient Guide*, §"NU5: the commitment becomes one of two").

10.5 Change and wallet posture

Consensus built the gate; wallets decide the traffic. ZIP-2005's posture is unambiguous: wallets SHOULD proactively move *all* funds — transparent, Sprout, and Sapling included, not merely Orchard — into the Ironwood pool, the only pool whose output notes are recoverable (§4; the analysis in §14.5), and SHOULD treat the sweep as an ongoing process rather than a one-off event.

Deployed change-routing logic implements the turnstile's grain, and is the routing rule the walkthrough of §8.1 deferred: change from Ironwood value stays in Ironwood; after activation, change may return to the Orchard pool only when the transaction spends Orchard notes *and* strictly less value returns than the spends remove; otherwise change is diverted to Ironwood (function `select_change_pool`, `librustzcash`, `zcash_client_backend/src/fees/common.rs`). The strict inequality keeps every such transaction on the right side of the seal's third rule: change never asks new value to enter the pool.

Scanning closes down symmetrically (ZIP-326). External Orchard receivers scan from the wallet birthday (§5.4) only up to NU6.3 activation — past it, compliant wallets no longer pay external Orchard receivers. Internal Orchard receivers scan until the wallet observes a *zero* Orchard balance after activation — change may still trickle back under the routing rule above. Ironwood receivers scan from the later of birthday and activation to the chain tip. The zero-balance watermark is sound because two seal rules act together, not because of non-negativity alone. The non-negativity rule closes the entry doors — no transaction moves new value into the pool — but an ordinary intra-pool payment satisfies it while raising its recipient's balance; what blocks that channel is the cross-address restriction: paying the wallet's Orchard receiver post-NU6.3 requires a spend at that same receiver, hence the wallet's own notes — none remain at balance zero — or its spending key (§10.3). With entry and circulation both shut, a wallet whose Orchard balance has reached zero can never see it become nonzero again (ZIP-326; ZIP-258).

10.6 Scheduled Orchard-to-Ironwood migration (ZIP-318)

ZIP-318 now supplies the wallet playbook that the consensus rules leave open. Its Draft recommends decomposing value into canonical denominations of the form $m\,10^k$ ZEC, where $m \in \{1, 2, 5\}$, from

0.01 through 10,000 ZEC, leaving any sub-0.01 ZEC residual unmigrated. For mobile wallets, preparation transactions SHOULD contain exactly 16 Orchard Actions. Each scheduled migration transaction then has one Orchard spend padded to two Actions, one unpadded Ironwood output, no transparent inputs or outputs, and the canonical fee (ZIP-318, "Creating Migration Transactions").

The Draft also coordinates timing so that individual wallets do not choose distinctive anchors and expiry heights. It provisionally draws anchors from a shared 144-block boundary grid, with a recency-weighted truncated distribution, and places expiry heights on a 34,560-block grid displaced by 69,120 blocks (ZIP-318, "Anchor Heights" and "Expiry Heights"). Wallets schedule and randomise broadcasts, offer network-layer privacy options, and obtain user consent before beginning. These wallet rules reduce the information disclosed by the public crossing amounts of §10.4; ZIP-229 and ZIP-258 remain the consensus-level authority for the crossing itself.

The upgrade is now fully on the table, and Table 2 collects it: one row per change, each with the place in the volume that constructs it. With the turnstile in place, the thesis journey of §1 closes — a zatoshi minted into the Ironwood pool lives the lifecycle of §§3–8, and one still resting in the sealed pool leaves through the gate just built. What the whole journey stands on — the circuit beneath the Action statement and the proofs behind each deferred proposition — is the work of §11 and §12.

NU6.3 change	Constructed in
Ironwood note format: lead byte `0x03`, hashed trapdoor `rcm`	Defs. 4.3, 4.4
Ironwood pool state: own tree (capacity 2^{32}), nullifier set, anchors	§6.5
Per-pool balancing field and binding signature	§7.2
Sighash: Ironwood branch, `_v6` personalisations, anchors to authorizing data	§7.3
Action statement: tenth public input and check A10	Defs. 8.1, 10.1
One circuit for both pools; verifying key selected by branch	§9.4
Shielding and coinbase doors re-scoped to Ironwood; maturity binds transparent outputs only	§9.1
The v6 format: Ironwood component; flags bit 2	§9.3
Ironwood chain value pool balance row	§§9.2, 10.4
Orchard pool sealed: three version-independent rules	§10.1
Fabricated outputs with random C_{enc} for restricted spends	§10.3
Ironwood anchor rule; three new history-tree fields	§10.4
Wallet: sweep posture, change routing, scan ranges	§10.5

Table 2: Everything NU6.3 changes, one row per change, with the part of the volume that constructs it (ZIP-229; ZIP-258). Consensus branch identity and activation constants: §9.3.

11 The circuit beneath the journey

Every stage of the journey deposited checks into the Action statement; this section descends one level, from statement to prover. It is deliberately a map, not a territory.

11.1 From relation to circuit

The Action statement of Definition 8.1 (§8.3) is a *relation*: it declares which (public input, witness) pairs are acceptable and prescribes no computation. What the prover executes is the *Action circuit* — a single fixed PLONKish circuit (the *Halo 2 Guide*, §"PLONKish arithmetisation"), identical for every Action, whose satisfiability Halo 2 proves in zero knowledge. The circuit encodes the conjunction A1–A10 and is supplied the statement's private witness: the note, the keys, the Merkle path, the randomiser α, and the value-commitment trapdoor `rcv`. What follows is a sketch of this encoding; gate-level detail is delegated to the *halo2 Book* and the *Orchard Book*.

The circuit is assembled from a small set of reusable *gadgets* (in the halo2 API, *chips*), each a pre-wired block of custom gates and lookups realising one operation. The dependency runs backwards through the volume: the Orchard designers chose the journey's primitives so that these gadgets would be inexpensive under PLONKish arithmetisation — why the journey met Sinsemilla and Poseidon rather than a bit-oriented hash (§2.4).

How any gadget is *built* — how its gates, lookups, and cell assignments are declared, and how synthesis turns the filled columns into the polynomials the proof commits to — is the subject of the *Halo 2 Guide*, §"From statement to circuit: arithmetisation in practice". This volume records only what each gadget *enforces*.

11.2 The gadget set

Five gadget families carry the ten checks.

ECC. The ECC gadget implements the Pallas group law: point addition by the complete addition formulas, with no exceptional cases (the *Math Guide*, §"Elliptic curves"); variable-base scalar multiplication as a double-and-add ladder of cheaper incomplete-addition rounds finished by a complete tail; fixed-base scalar multiplication from precomputed tables; and witnessing that a point lies on the curve. (The j-invariant-0 endomorphism — the *Math Guide*, §"The j-invariant and the GLV endomorphism" — serves only the recursion layer (the *Halo 2 Guide*, §"Accumulation and the Halo trick"), cheapening multiplication by verifier challenges; the Action circuit does not use it.) The gadget realises every group operation in the statement: $\mathsf{ak} = [\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$ and $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ (A6); $\mathsf{pk_d} = [\mathsf{ivk}]\, \mathsf{g_d}$ (A7); $\mathsf{cv}^{\mathsf{net}} = [v_{\mathrm{old}} - v_{\mathrm{new}}]\, V^{\mathsf{Orchard}} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}}$ (A4); and the point $[F_{\mathsf{nk}}(\rho) + \psi]\, G^{\mathsf{nf}} + \mathsf{cm}$ inside the nullifier (A5).

Sinsemilla. The Sinsemilla gadget implements Construction 2.1 by realising the generator table $P[m]$ as a Halo 2 lookup (the *Halo 2 Guide*, §"The lookup argument"): each 10-bit chunk costs one table lookup and two incomplete additions (§2.4). On it rest the two composed commitment gadgets the *Orchard Book* names: NoteCommit, proving the note commitments of A1 and A2, and $\mathsf{Commit}^{\mathsf{ivk}}$, the in-circuit short commitment proving $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract(ak)}, \mathsf{nk})$ in A7 — each bundling the Sinsemilla rounds with the bit-decomposition and canonicity checks of its field inputs.

Merkle path. The Merkle-path gadget stacks 32 layer-tagged $\mathsf{MerkleCRH}^{\mathsf{Orchard}}$ hashes — themselves Sinsemilla short hashes (Definition 6.1). At each layer a conditional swap orders the running node and its witnessed sibling by one bit of the leaf position; the gadget asserts the recomputed root equal to the public anchor — check A3, gated by v_{old} so that a dummy spend may skip it.

Poseidon. The Poseidon gadget implements the Poseidon permutation (the *Crypto Guide*, §"Arithmetisation-friendly hashing: Poseidon"), whose only non-linearity is $x \mapsto x^5$, to evaluate the nullifier PRF $F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho)$ of Definition 7.1 cheaply in-circuit; its output feeds the ECC gadget for A5.

Decomposition. Lookup-based decomposition, range, and boolean gadgets split field elements into the bit-chunks the other gadgets consume and range-check witnessed values: $v_{\mathrm{old}}, v_{\mathrm{new}} \in [0, 2^{64})$ and $\mathrm{sign} \in \{-1, +1\}$ (A4), and canonical point and field-element encodings.

11.3 Flags, public inputs, and circuit parameters

The flags enableSpends, enableOutputs, and enableCrossAddress (bit 2) need no gadget at all. They are verifier-supplied public inputs: ZIP-229's flags bytes, flagsOrchard and flagsIronwood, fix them to bits outside the circuit, and they enter it only through the plain multiplication gates of A8–A10 — the

third negated, wired as the Ironwood circuit's tenth public input disableCrossAddress (Definition 8.1; semantics in §10.2).

The gadgets compose into one fixed circuit whose native field is the Pallas base field $\mathbb{F}_{p_{\mathsf{Pallas}}}$ — the field of Pallas coordinates, so the ECC gadget runs natively — proved with the Halo 2 inner-product argument on Vesta, whose scalar field the Pasta cycle makes exactly $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (the *Halo 2 Guide*, §"The inner-product polynomial commitment"). The circuit occupies $2^{11} = 2048$ rows — the *Halo 2 Guide*'s circuit-size exponent $k = 11$, not Sinsemilla's chunk width $k = 10$ (Construction 2.1) — with ten advice columns beside the fixed columns and the Sinsemilla lookup table (the *Halo 2 Guide*, §"From statement to circuit: arithmetisation in practice"). Every Action, whatever it spends or creates, is one instance of this same circuit; a bundle's Actions are proved together in the one aggregate proof a node checks from the public inputs alone (§8.2); and batch verification — the deferred linear multi-scalar multiplications merged across proofs (the *Halo 2 Guide*, §"Accumulation and the Halo trick") — lets a node verify many proofs at amortised cost approaching a single verification. The machinery the journey met one stage at a time is, beneath it all, a single two-thousand-row table whose satisfiability is the payment's validity.

12 End-to-end security

The journey's debts now come due. Each property was stated where its object was constructed and deferred here: Sinsemilla collision resistance (Theorem 2.2, §2.3), nullifier uniqueness (Proposition 7.2, §7.1), RedPallas unforgeability under re-randomisation (Proposition 7.5, §7.4), and the balance equivalence (Proposition 7.4, §7.2). This section pays them in order, adds the two properties with no single construction site — zero knowledge and the soundness of the composed whole — and closes with the volume's final theorem.

12.1 Guarantees provided by a valid Action proof

A SNARK π accepted on the public inputs

$$(\mathsf{anchor},\ \mathsf{cv}^{\mathsf{net}},\ \mathsf{nf}_{\mathsf{old}},\ \mathsf{rk},\ \mathsf{cmx},\ \mathsf{enableSpends},\ \mathsf{enableOutputs},\ \mathsf{disableCrossAddress})$$

testifies, with overwhelming probability, that the prover *knows* a witness satisfying the checks A1–A10 of Definition 8.1. This is the knowledge soundness of Halo 2 (the *Halo 2 Guide*, §"A rigorous treatment of knowledge soundness"): from any prover that convinces the verifier, an extractor recovers such a witness. Everything on the soundness side — nullifier uniqueness, spend authority, balance — then follows from one or more conjuncts of A1–A10 together with algebraic properties of the primitives those conjuncts invoke, and §12.2 supplies the primitive reduction they all share. Privacy rests instead on the zero knowledge of the SNARK (§12.6) together with the DDH, Poseidon-PRF, and KDF/AEAD assumptions that Theorem 12.3(iv) adds.

12.2 Sinsemilla collision resistance

Proposition 12.1 (Sinsemilla collisions are discrete-log relations)**.** *A collision $M \neq M'$ with $\mathsf{Sinsemilla}_D(M) = \mathsf{Sinsemilla}_D(M')$ yields an explicit non-trivial discrete-log relation among the random-oracle-derived generators $Q(D), P[0], \ldots, P[2^k - 1]$ of Construction 2.1; so does an x-coordinate collision $\mathsf{ShortHash}_D(M) = \mathsf{ShortHash}_D(M')$, and, with the blinding base H_D joining the generator set, a collision in* ShortCommit *or in the extracted commitment* cmx*. Consequently Theorem 2.2 holds — for fixed-length inputs, $\mathsf{Sinsemilla}_D$ is collision-resistant assuming DL on $\mathbb{G}$ — and the short forms of Definition 2.3 inherit it.*

Sketch. Suppose first that both messages have the same chunk count ℓ and that no incomplete-addition exception occurs. Unfolding the update rule of Construction 2.1,

$$\mathsf{Sinsemilla}_D(M) \;=\; [2^\ell]\,Q(D) \;+\; \sum_{i=1}^{\ell} \big[2^{\ell-i}\big]\,P[m_i]. \tag{1}$$

A collision therefore gives $\sum_i [2^{\ell-i}]\,(P[m_i] - P[m_i']) = \mathcal{O}$ with $m_i \neq m_i'$ for at least one i: a non-trivial linear relation over $\mathbb{F}_{p_{\mathsf{Vesta}}}$ on the table generators. Since hash-to-curve modelled as a random oracle samples the generators independently (§2.2), finding such a relation is the multi-generator discrete-log-relation problem, and no efficient adversary produces one without solving DL — the reduction the *Crypto Guide* gives in its Pedersen-hash collision analysis, §"Sinsemilla: an algebraic hash-based commitment". (The fixed-length hypothesis is load-bearing, not a convenience: the deployed hash zero-pads its input to a chunk multiple, Construction 2.1, so two messages of different bit-lengths agreeing after padding collide with no discrete-log relation at all — variable-length collision resistance is simply false.) An x-coordinate collision adds one further case: $\mathsf{Extract}$ identifies P with $-P$ (§2.1), so $\mathsf{Extract}(\mathsf{Sinsemilla}_D(M)) = \mathsf{Extract}(\mathsf{Sinsemilla}_D(M'))$ forces $\mathsf{Sinsemilla}_D(M) = \pm\mathsf{Sinsemilla}_D(M')$. The $+$ case is the point collision just treated; in the $-$ case, moving everything to one side of equation (1) gives

$$[2^{\ell+1}]\,Q(D) + \sum_{i=1}^{\ell} \big[2^{\ell-i}\big]\,\big(P[m_i] + P[m_i']\big) = \mathcal{O},$$

whose $Q(D)$-coefficient $2^{\ell+1}$ is non-zero modulo p_{Vesta}: a non-trivial relation whether or not any chunk differs. The same one extra term extends the argument to the blinded forms — adding further independent bases, H_D for $\mathsf{ShortCommit}$ and the extracted cmx, leaves the relation non-trivial. Finally, the incomplete-addition exceptions — inputs on which some $\dot{+}$ is undefined — are equalities between accumulator points, themselves non-trivial discrete-log relations among the same generators, so exhibiting one is negligible too — the step that priced the $\perp$-weakening of Remark 8.2. □

Collision resistance is the primitive ingredient of note-commitment binding: since the blinding base H_D of $\mathsf{NoteCommit}$ is independent of the hash generators (§2.3), opening one commitment to two distinct notes — even at the level of the extracted cmx, by the $\pm$ case above — would again exhibit a relation Proposition 12.1 excludes: two distinct notes cannot share a commitment, nor a cmx. Checks A1 and A2 are therefore genuine integrity statements about $\mathsf{cm}_{\mathrm{old}}$ and the new note's cmx, and with knowledge soundness they entail that the prover knows the *specific* opening of each commitment — the fact every argument below consumes.

12.3 Nullifier uniqueness

The extraction just established discharges the double-spend property: two distinct notes share a nullifier only with probability negligible in the DL hardness of Pallas, with the derivation of G^{nf} modelled as a random oracle (§2.2) — exactly the hypotheses Proposition 7.2 names. No pseudorandomness of F is invoked, nor could it be: the adversary who forges the colliding notes holds the PRF key nk itself, and every coefficient below, $F_{\mathsf{nk}}(\rho)$ included, is known to the reduction.

Sketch of Proposition 7.2. A collision $\mathsf{nf}(\mathbf{n}) = \mathsf{nf}(\mathbf{n}')$ for notes $\mathbf{n} \neq \mathbf{n}'$ is an equality of x-coordinates (Definition 7.1), hence

$$\big[\,F_{\mathsf{nk}}(\rho) + \psi\,\big]\,G^{\mathsf{nf}} + \mathsf{cm} \;=\; \pm\Big(\big[\,F_{\mathsf{nk}'}(\rho') + \psi'\,\big]\,G^{\mathsf{nf}} + \mathsf{cm}'\Big),$$

a non-trivial linear relation among G^{nf}, cm, and cm'. Expanding cm and cm' through their known openings — extracted by knowledge soundness, or supplied by the adversary who forged the notes — over the Sinsemilla generators and the blinding base H_D turns this into a *known* non-trivial discrete-log relation among the independent $\mathsf{GroupHash}$ outputs $\{G^{\mathsf{nf}}, Q, P[0], \ldots, P[2^k - 1], H_D\}$: distinctness

of the notes forces a coefficient mismatch somewhere. Producing such a relation reduces to DL on Pallas exactly as in Proposition 12.1. □

Check A5 (nullifier integrity) plus knowledge soundness then ensures that the nf_{old} among the public inputs is genuinely *the* nullifier of a specific committed note, not an arbitrary value; the chain's duplicate-nullifier rejection (§8.2) does the rest.

12.4 RedPallas unforgeability

The signature scheme's debt, Proposition 7.5, carries a sharpened reading: a forger that, given ak and signatures under re-randomised keys, produces a valid signature under $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ for an α of its own — possibly ak-dependent — choice yields a DL solver for ak.

Sketch of Proposition 7.5. The standard forking argument (the *Crypto Guide*, §"Security in the random oracle model and the forking lemma", via the Bellare–Neven general forking lemma) reduces a Schnorr forger to a DL adversary, and it covers re-randomisation because RedPallas is *key-prefixed*: the challenge $H^*(R \,\|\, \mathsf{rk} \,\|\, m)$ hashes the verification key, so shifting responses cannot transport a signature from one key to another. The reduction embeds the DL challenge as $\mathsf{ak} := X$. It answers signing queries under any $\mathsf{rk}_i = \mathsf{ak} + [\alpha_i]\, G^{\mathsf{SpendAuth}}$ without knowing any secret, via the honest-verifier zero-knowledge simulator with random-oracle programming: sample c, s uniformly, set $R := [s]\, G^{\mathsf{SpendAuth}} - [c]\, \mathsf{rk}_i$, and program $H^*(R \,\|\, \mathsf{rk}_i \,\|\, m) := c$, aborting on programming collisions exactly as in the Schnorr EUF-CMA theorem of the same section. It then forks the adversary at the forgery's critical query $H^*(R^* \,\|\, \mathsf{rk}^* \,\|\, m^*)$, obtaining two accepting transcripts that share R^* — and, since both forks share the critical query's input, the same rk^*, hence the same α^* — with distinct challenges. Two-special soundness (the *Crypto Guide*, §"The Schnorr identification protocol") extracts rsk^*, the discrete logarithm of rk^*, and the output $\mathsf{ask} = \mathsf{rsk}^* - \alpha^*$, with α^* the adversary-supplied randomiser, solves DL for ak. □

A separate, weaker statement covers honest signers only: for uniform signer-chosen α, sampling α at random and back-deriving $\mathsf{ak} := \mathsf{rk} - [\alpha]\, G^{\mathsf{SpendAuth}}$ reproduces the joint view of $(\mathsf{ak}, \alpha, \mathsf{rk})$ exactly, so honest re-randomisations are distributed as plain Schnorr keys — the unlinkability statement of the *Crypto Guide*, §"Key re-randomisation and unlinkability", which does not by itself cover adversary-chosen α; the forking reduction above is what does.

Check A6 (spend authority) plus knowledge soundness ensures the prover knows the α relating rk to ak; combined with the SpendAuthSig under rk at the transaction layer (§7.4), this enforces that the spender knows the ask behind ak — and hence, through check A7's binding of ak into the address, the authority over $\mathsf{pk}_{\mathsf{d}_{\text{old}}}$.

12.5 Balance preservation

Proposition 12.2 (Balance). *Any accepted Orchard bundle satisfies*

$$\sum_i \left(v_{\text{old},i} - v_{\text{new},i}\right) = v_{\text{balance}}^{\mathsf{Orchard}},$$

except with probability negligible in the DL hardness of the Pasta curves in the random-oracle model — Vesta for the knowledge soundness of the Action SNARK supplying the A4 openings, Pallas for the binding-signature extraction and the independence of the value-commitment bases.

Sketch. By A4 and knowledge soundness, each $\mathsf{cv}^{\mathsf{net},i}$ commits to the true per-Action difference $v_{\text{old},i} - v_{\text{new},i}$, and by the Pedersen homomorphism the bundle sum $\sum_i \mathsf{cv}^{\mathsf{net},i}$ splits into a V^{Orchard}-component carrying $\sum_i (v_{\text{old},i} - v_{\text{new},i})$ and an R^{Orchard}-component carrying $\sum_i \mathsf{rcv}_i$ (§7.2). Consensus computes

bvk and checks the binding signature; a valid binding signature is a Fiat–Shamir proof of knowledge of $\log_{R^{\mathsf{Orchard}}} \mathsf{bvk}$ (the *Crypto Guide*, §"Binding signatures as a proof of knowledge of a discrete logarithm"), so the forking extraction from the proof of Proposition 7.5 recovers bsk with $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$. Combined with the extracted A4 openings, a non-zero V^{Orchard}-component of bvk would yield a discrete-log relation between the independent bases V^{Orchard} and R^{Orchard}, contradicting DL. Hence the value component equals $[v^{\mathsf{Orchard}}_{\text{balance}}]\,V^{\mathsf{Orchard}}$ and the total value flow matches the declared transparent balance. This also completes Proposition 7.4: the "if" direction was algebraic at the construction site, and the extraction just given shows that an unbalanced author cannot know a discrete logarithm of bvk to base R^{Orchard}. □

Balance preservation is the fundamental conservation property: shielded transactions cannot mint zatoshi out of nothing.

12.6 Zero knowledge

Halo 2 is *statistical* zero knowledge in the random-oracle model (the *Halo 2 Guide*, §"Zero knowledge: hiding the witness in Halo 2"). On input the instance $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\mathsf{old}}, \mathsf{rk}, \mathsf{cmx}, \ldots)$ — and no witness — the simulator operates as follows:

1. sample the Fiat–Shamir challenges $\theta, \beta, \gamma, y, x, x_1, x_2, x_3, x_4$ and the inner-product argument (IPA) challenges $u_1, \ldots, u_k$ ($k = 11$ rounds, the circuit-size exponent of the 2^{11}-row circuit, §11) uniformly, programming the random oracle to return them on the relevant transcript prefixes;

2. sample the final IPA scalar a uniformly;

3. use the freedom in the blinding terms l_j, r_j added at each IPA round and in the polynomial blinders added to each committed prover polynomial — the advice columns in phase 1, the running products in phase 3, the quotient chunks in phase 4 — to satisfy the verifier's equations algebraically, without ever knowing the witness.

The resulting transcript is statistically indistinguishable from a real proof; the only gap from *perfect* zero knowledge is the random-oracle programming, which a verifier that observes only the transcript cannot detect.

Zero knowledge supplies the proof's share of Orchard's privacy: the SNARK transcript reveals nothing beyond the public inputs of each Action. That the published data leak nothing *further* is a separate, computational matter: the nullifier — itself a public input (§7.1) — hides its note under the PRF security of Poseidon or, independently, DDH on Pallas (the disjunction stated there), and the epk and ciphertexts published alongside the public inputs (§5) hide recipient and value under the DDH and KDF/AEAD assumptions that Theorem 12.3(iv) adds.

12.7 Composition: the end-to-end theorem

The full soundness of the Action SNARK composes three ingredients.

- *PIOP soundness.* Each polynomial-identity check of the underlying polynomial interactive oracle proof (PIOP; the *Halo 2 Guide*, §"Polynomial IOPs and the SNARK design pattern") carries a Schwartz–Zippel error (the *Math Guide*, §"Polynomials over a field"), summed across constraints. For $\mathbb{F} = \mathbb{F}_{p_{\mathsf{Pallas}}}$ with $|\mathbb{F}| \approx 2^{254}$ and constraint degree at most $d_{\max}$, the error per check is at most $d_{\max}\, n/|\mathbb{F}| \approx d_{\max} \cdot 2^{-243}$: the gate polynomial composes degree-$d_{\max}$ gates with column polynomials of degree $n-1$ for $n = 2^{11}$ rows, so a cheating prover's recombined quotient $h \cdot Z_H$ has degree below $d_{\max}\, n$. The bound is overwhelmingly small.

- *PCS extractability.* Extraction for the inner-product commitment reduces to DL on the IPA curve, Vesta (the *Halo 2 Guide*, §"The inner-product polynomial commitment"), with the accumulation

analysis (the *Halo 2 Guide*, §"Accumulation and the Halo trick") covering the deferred-MSM batching by which a node amortises verification across proofs (§11); the same analysis would carry soundness across recursion, which Orchard's deployment does not use.

- *Fiat–Shamir soundness in the random-oracle model (ROM).* The Fiat–Shamir theorem of the *Crypto Guide*, §"The Fiat–Shamir transform: from interactive to non-interactive", covers Σ-protocols, losing roughly a factor Q in the prover's random-oracle query budget; for the $\log d$-round Halo 2 transcript the naive multi-round analysis loses $Q^{O(\log d)}$, a bound vacuous already at $Q = 2^{80}$. The negligible-error conclusion rests on the tighter treatments the *Halo 2 Guide* presents in §"The algebraic group model": the direct ROM-plus-DL analysis of BCMS20, or Ghoshal–Tessaro's tight Fiat–Shamir bound in the AGM.

Each step's error is negligible at Orchard's parameters: $|\mathbb{F}| \approx 2^{254}$, security parameter $\lambda = 127$ under the convention that the group order is about $2^{2\lambda}$ — effectively about 126 bits after the automorphism speedups (the *Crypto Guide*, §"Instantiation on elliptic curves; the Pasta curves") — and the union bound across the steps is itself negligible. Combined with the four application-level reductions — Propositions 12.1, 7.2, 7.5, and 12.2 — the full chain yields the volume's final theorem.

> **Theorem 12.3** (Orchard end-to-end security, informal)**.** *Under DL hardness on both Pasta curves (Pallas for the application-level primitives, Vesta for the IPA commitment) in the random-oracle model — and, for property (iv), additionally DDH on Pallas, the PRF security of Poseidon (§7.1), and the KDF/AEAD securing the note ciphertext (§5) — no efficient adversary can:*
>
> (i) *produce an accepted Action without knowing a witness satisfying A1–A10; in particular, any Action whose witnessed v_{old} is non-zero must spend a genuine leaf of the commitment tree (zero-value dummy spends, admitted by A3's gate, are accepted by design);*
>
> (ii) *cause two Actions to share a nullifier without spending the same note twice;*
>
> (iii) *cause a bundle's declared transparent balance $v_{\text{balance}}^{\text{Orchard}}$ to differ from its true net value flow $\sum_i (v_{\text{old},i} - v_{\text{new},i})$ — in particular, to exceed it, which would create value (Proposition 12.2);*
>
> (iv) *link a published Action to any specific recipient or value beyond what is explicit in the public inputs.*

The four properties are, respectively, the soundness of the Action SNARK, nullifier uniqueness, balance preservation, and zero knowledge — and they discharge the four requirements of §1: property (i) covers R1 and R2 (value moves only through well-formed notes with proven membership), (ii) is R3, (iii) is R4, and (iv) is the disclosure clause over them all. The first three reduce, informally, to DL on the Pasta curves in the random-oracle model — Pallas for the application-level reductions, Vesta for the IPA extraction. For the fourth, the zero knowledge of the SNARK transcript is *statistical*: given the perfectly-hiding Pedersen and Sinsemilla blinders and the random-oracle programming, it does not rest on DL. Full unlinkability of an Action, however, additionally rests on the DDH assumption on Pallas, the PRF security of Poseidon (§7.1), and the KDF/AEAD securing the note ciphertext (§5), and is therefore computational — the typical asymmetry of the discrete-log SNARK family: soundness is computational (a cheating prover is bounded, never impossible), while the zero knowledge of the transcript is information-theoretic. The journey's debts are paid; §13 sets these proofs in their history.

13 Lineage: three generations of shielded proof

The machinery whose security §12 has just established did not spring up whole: Zcash has carried shielded payments through three generations of proof system, and concrete trade-offs drove each transition — trusted-setup risk, recursion compatibility, post-quantum posture, and on-chain efficiency. This closing survey sets the volume's stack in that history.

13.1 Sprout (2016–present, deprecated)

The original shielded protocol proved its statements with a *BCTV14* SNARK (Ben-Sasson, Chiesa, Tromer, Virza) over the pairing-friendly curve BN254 — a scheme of the pairing-based family the *Crypto Guide* surveys in §"SNARKs: succinct non-interactive arguments of knowledge" — with constant-size proofs (296 bytes) and constant verification time. Its parameters came from a per-circuit trusted setup: the 2016 ceremony, a six-party multi-party computation generating the BCTV14 proving and verification keys for the fixed Sprout circuit, sound provided at least one participant destroyed their secret share. Sprout demonstrated that zk-SNARK shielded payments were feasible at scale, but it remained a transitional design: the prover was slow, and the circuit hashed with SHA-256 — a bit-oriented hash whose boolean operations are non-native to arithmetic constraints, hence costly in-circuit. Decisively, a flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; the pool was deprecated, and shielded Sprout value was migrated to later pools.

13.2 Sapling (2018–present)

The second network upgrade, Sapling, replaced this stack with Groth16 — 192-byte proofs, three-pairing verification — over BLS12-381, with the embedded curve Jubjub serving the in-circuit group operations and a Pedersen hash (the *Crypto Guide*, §"Commitment schemes") serving the Merkle tree. The result was a far faster prover, a smaller proof, and cheaper in-circuit hashing. Two structural limitations remained. Groth16 still requires a per-circuit trusted setup — the 2018 Sapling MPC, whose universal first phase was the Powers of Tau ceremony with many participants — and no known elliptic-curve cycle extends BLS12-381 (the *Halo 2 Guide*, §"The necessity of a cycle of curves"), so the proving system admits no efficient recursion.

13.3 Orchard and the Pasta cycle (2022–present)

Orchard, activated by NU5 in 2022 and running unchanged in the Ironwood pool from NU6.3, replaced the Sapling stack with the Halo 2 / Pasta stack this volume has used throughout. Four choices are principal.

- *Halo 2*: a PLONKish PIOP compiled with the IPA polynomial commitment, accumulation-friendly, and with no trusted setup (the *Halo 2 Guide*, §"The Halo 2 proof system").
- *The Pasta cycle*: Pallas as the application curve, Vesta as the proving curve in the IPA layer, designed for recursion (the *Halo 2 Guide*, §"Recursive proof composition and accumulation schemes").
- *Sinsemilla*: an algebraic hash whose collision resistance reduces to DL on Pallas — an assumption the protocol already requires elsewhere — and whose chunk-table structure maps directly onto Halo 2's lookup argument (§2.3).
- *Poseidon*: serves the nullifier PRF (§7.1), where a keyed in-circuit PRF rather than a collision-resistant hash is required; unlike Sinsemilla, its security rests on cryptanalytic confidence in algebraic-attack resistance rather than on a hardness assumption such as DL.

The shift followed one consistent design principle: "transparent SNARK, recursion-ready" was the target, and the designers engineered the Pasta cycle to fit it.

The stack has since carried one correction and one split, and neither touched its cryptography. The Action circuit exists in three versions — NU5's original, the NU6.2 emergency correction (ZIP-257, Final), and the NU6.3 Ironwood circuit with the cross-address input (§10.2) — named `InsecurePreNu6_2`, `FixedPostNu6_2`, and `PostNu6_3` in the released crate `orchard` 0.15.0 (`src/circuit.rs`; the `bef8a27` development tree names the third variant `Ironwood`); the operational list is Remark 9.1 (§9). And from NU6.3 the one protocol carries two value pools, the sealed Orchard pool and the current Ironwood pool (ZIP-229, Terminology; §9). The Halo 2 / Pasta stack is unchanged throughout.

13.4 Comparative summary

Table 3 gathers the three generations.

	Sprout	Sapling	Orchard
SNARK	BCTV14	Groth16	Halo 2
Curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
Trusted setup	Per-circuit (six-party MPC)	Per-circuit (large MPC)	None
Recursion-ready	No	No	Yes (via Pasta cycle)
Proof size	296 B	192 B	∼5 kB (one Action)
Verification time	Constant (pairing)	Constant (3 pairings)	$O(\log n)$ amortised

Table 3: Three generations of Zcash shielded-payment proof systems. Orchard verification is $O(\log n)$ amortised ($n = 2^{11}$ rows): the per-proof checks are succinct, and the linear multi-scalar multiplication is batched across proofs (the *Halo 2 Guide*, §"Accumulation and the Halo trick"). The Orchard column covers three circuit versions — the NU5 original; the NU6.2 correction (ZIP-257, Final); the NU6.3 Ironwood circuit with the cross-address input — and, from NU6.3, two value pools (§9).

The proof size grew between Sapling and Orchard, and the growth is the cost of transparency: a one-Action bundle's proof occupies $2720 + 2272 = 4992$ bytes in the v5/v6 encodings (Table 1, §9), including the $2\log_2 2^{11} = 22$ group elements of the IPA rounds. The benefit, beyond the removal of trusted setup, is the foundation for recursion: future generations — the Tachyon scalability programme (§14.6), and eventual designs that aggregate many transactions under a single succinct proof — can build on Orchard's foundation without revisiting the cryptographic stack. Of the opening list's four trade-offs, the post-quantum posture alone remains unpriced; it is the horizon of §14.

14 The horizon: quantum adversaries and recoverability

The lineage of §13 looked backward; this closing section looks forward, and asks the one question the thesis journey of §1 leaves open: whether the value survives its owner's adversaries. A *cryptographically relevant quantum computer* (CRQC) — one large and reliable enough to run Shor's algorithm at cryptographic parameters (the *Crypto Guide*, §"The quantum caveat: Shor's algorithm") — computes discrete logarithms in polynomial time, and with them falls every group-structured assumption of the end-to-end theorem: DL on both Pasta curves and, for property (iv), DDH on Pallas (Theorem 12.3). The theorem's remaining assumptions — the PRF security of Poseidon and the security of the KDF/AEAD — are symmetric, and face only the generic quantum speedups of §14.4.

14.1 The reversibility taxonomy and the epistemic bins

Since the assumptions all fail together, asking *whether* they fail is unproductive. The productive question is when the failure matters, and the axis that answers it is reversibility.

> **Definition 14.1** (Reversibility of a quantum break)**.** A quantum break of a deployed protocol is *retroactive* when data already recorded on chain becomes exploitable on the day the adversary exists: no later consensus change can help, because the data is public and permanent. A break is *prospective* when its exploitation requires a validator to *accept* a forged object after the adversary exists: disabling the affected protocol before that day removes every future acceptance event and mitigates the break completely.

Classified along this axis, the Orchard protocol — both its pools — has exactly one retroactive break: harvest-now-decrypt-later against the note encryption (§14.2). Everything else on its assump-

tion surface is prospective — the discrete-log integrity stack of §14.3 — or faces only generic speedups — the symmetric primitives of §14.4. The section is organised accordingly, and its scope is the Orchard core seen from inside: the same reversibility axis applied across every layer of Zcash — the transparent stack, proof of work, the frontier protocols, and the migration arithmetic — is the *PQ Guide*'s subject. That volume is a lateral companion; nothing below depends on it.

Remark 14.2 (Method)**.** Every forward-looking claim below sits in one of the three epistemic bins of §9: *specified* — a published artefact pins the construction down, and is cited; *designed but unspecified* — an informal source describes the design, and the text says what a specification would still need to fix; *open problem* — no artefact exists. Deployed-behaviour claims instead cite implementation and specification, as throughout the volume. The ground truth is pinned: `protocol.tex` and the ZIPs at `zips` commit `69610984` (2026-07-09); crate `orchard` at git commit `bef8a27e` (v0.14.0, 2026-06-16) for circuit claims; the released `orchard` 0.15.0 — the `librustzcash` pin — for the lead-byte-`0x03` note path. Each artefact cited below was verified at these pins.

14.2 The retroactive break: harvest now, decrypt later

Each Action publishes the pair $(\mathsf{epk}, C_{\mathrm{enc}})$ (§5). The confidentiality of C_{enc} rests on exactly one discrete-log-dependent link: the one-shot Diffie–Hellman on Pallas between esk and $\mathsf{pk_d}$ (Construction 5.1; protocol § 5.4.5.5; crate `orchard`, `src/note_encryption.rs`, with the esk derivation in `src/note.rs` and the key agreement in `src/keys.rs`). The KDF and the AEAD above that link are symmetric, and are not the weak layer.

The attack this admits is *harvest now, decrypt later* (HNDL): record the chain today, break the link when the machine arrives. A CRQC that knows a recipient's address $(d, \mathsf{pk_d})$ computes $\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d}$ by Shor — one discrete logarithm for the lifetime of the address — and then runs the recipient's own trial-decryption loop of §5.3 over the recorded chain: for every Action, $[\mathsf{ivk}]\,\mathsf{epk}$ recovers K_{enc}, and the AEAD tag identifies which ciphertexts belong to the address. The adversary obtains, for every note ever sent to that address, the full plaintext $(d, v, \mathsf{rseed}, \mathsf{memo})$: every amount, every memo, the complete receiving history — and equally every *future* ciphertext to the same address.

Two limits bound the damage. First, ivk is a viewing capability, not a spending one (the ladder of §3.4): no spend authority is conferred, and ask is not exposed — its exposure is a prospective matter, taken up in §14.3. Second, the attack is keyed by the address: without $(d, \mathsf{pk_d})$ the adversary has neither the base $\mathsf{g_d}$ nor the target $\mathsf{pk_d}$, and the recorded chain reveals nothing further even to an unbounded adversary (§14.3 completes this claim). ZIP-2005 records the same boundary: the exposure requires "both the ciphertext ... and the receiver address".

The break is retroactive by construction. The pair $(\mathsf{epk}, C_{\mathrm{enc}})$ is consensus data, replicated on every full node forever; its secrecy was fixed at recording time by one specific Diffie–Hellman instance; and any protocol upgrade alters only ciphertexts not yet created (ZIP-2005, Non-requirements). The adversary needs no quantum computer today — only storage — and every Orchard output recorded since NU5 activation is already in that corpus, which accrues Ironwood outputs from NU6.3 activation onward.

Bin: *open problem*. Nothing is deployed, and nothing is specified, that protects even future ciphertexts. ZIP-2005 (Proposed) excludes privacy by an explicit Non-requirement, states that the exposure persists for all pools — Ironwood included — and says only that mitigations for future transfers are "under consideration". The tracking issues `zcash/zips#1133` (open since 2022) and `zcash/zips#1134` (open since 2016) carry discussion, not drafts; and the zips repository at commit `69610984` contains no occurrence of ML-KEM or its pre-standardisation name Kyber (the NIST post-quantum key encapsulation), of ML-DSA (the corresponding signature standard), or of any lattice scheme. A future hybrid key encapsulation would in any case protect future outputs only; the recorded ciphertexts are unrepairable.

14.3 Prospective breaks: the discrete-log integrity stack

The security reductions of §12 each run equally well in reverse: the assumption a CRQC falsifies is exactly the one the corresponding proof consumes, so the prospective-break inventory reads directly off that section's reduction list. We take its four entries in construction order.

Sinsemilla binding. Collision resistance reduces to the discrete-log relation problem among the GroupHash-derived generators (Proposition 12.1). A CRQC computes every pairwise logarithm, after which each Sinsemilla commitment opens to arbitrary messages: a second note under an on-chain cmx — breaking balance through NoteCommit non-binding — a fabricated membership path under any anchor (§6), and alternative pairs $(\mathsf{ak}', \mathsf{nk}')$ opening the same $\mathsf{Commit}^{\mathsf{ivk}}$ — the "Spendability" attacks ZIP-2005 catalogues, forgeries of the kind the Faerie resistance of §7.1 excludes. A balance violation committed this way is bounded only by the ZIP-209 turnstile (§10.4) and need not be detected at all.

Value commitments and the binding signature. The binding of $\mathsf{cv}^{\mathsf{net}}$ is the unknown discrete-log relation between the bases V^{Orchard} and R^{Orchard} (Definition 7.3, Proposition 7.4), and the binding signature is Schnorr under DL in the random-oracle model. Knowledge of $\log_{V^{\mathsf{Orchard}}} R^{\mathsf{Orchard}}$ reopens any recorded or fresh cv to any value and yields a bsk for any target bvk: silent inflation, again turnstile-bounded (crate `orchard`, `src/value.rs`).

Spend authorisation. RedPallas unforgeability is DL in the random-oracle model (Proposition 7.5). One logarithm — of ak, or of any published rk — forges spend authorisation at will; combined with the proof-system break below it completes arbitrary theft (crate `orchard`, `src/primitives/redpallas.rs`).

Proof soundness. The Action SNARK is knowledge-sound under DL on Vesta with Fiat–Shamir in the random-oracle model (§12.7; crate `halo2_proofs`, `src/poly/commitment.rs`). A CRQC forges accepting proofs of false Action statements — minting from nothing, spending any commitment on chain — indistinguishable from honest proofs. This is the single most concentrated failure point: every other item's exploitation route runs through it.

None of the four is retroactive, and the reason deserves spelling out. Each attack must place a forged object — a proof, a signature, a commitment opening — before a validator and have it *accepted*, and acceptance happens at current height under current consensus rules. Disabling the DL-dependent pools before a CRQC exists therefore removes every future acceptance event; transactions already recorded were validated when the assumptions held, and are never re-adjudicated. Forking in time is a complete mitigation for the integrity stack — with two residues. First, switch-off must precede the first forgery: an attack landed inside the exposure window is not repaired afterwards. Second, honest funds inside a disabled pool need some other way out — which is precisely the problem ZIP-2005 exists to solve (§14.5).

The mitigation is complete in a second sense: the recorded chain does not leak, even quantumly, the material the integrity attacks would need. The value commitment is perfectly hiding, so recorded cv values reveal nothing about amounts even to an unbounded adversary (§7.2); the randomiser α is uniform, so recorded rk values and signatures carry information-theoretically no linkage to ak or ask (§12.4); proof transcripts are statistically simulatable without the witness (§12.6); and nullifiers are keyed by nk, which never appears on chain (§7.1). A quantum adversary who does not know a recipient's address consequently learns nothing from the recorded chain — ZIP-2005 reaches the same conclusion — and the sole retroactive channel is the address-keyed decryption of §14.2.

14.4 Primitives facing only generic speedups

Two primitive families in Orchard carry no discrete-log structure. BLAKE2b serves PRFexpand and with it every key and note-randomness derivation (Definition 3.2, §3; Definition 4.3), the KDF and the outgoing cipher key (§5), the RedPallas challenge hash, and the ZIP-244 sighash (§7.3); the ZIP-32 (Final) Orchard key tree is hardened-only, built entirely from these derivations, with no elliptic-curve operation anywhere on the derivation path (crate `orchard`, `src/zip32.rs`). Poseidon serves

the nullifier PRF, keyed by nk (§7.1; crate `orchard`, `src/note/nullifier.rs`).

Against these a quantum adversary gets generic speedups and nothing more. Grover's search algorithm halves effective preimage security: 2^{256} classical evaluations become about 2^{128} quantum ones, a margin the parameters absorb (the *Crypto Guide*, §"The quantum caveat: Shor's algorithm"). For collisions the picture is better still: the Brassard–Høyer–Tapp (BHT) algorithm finds collisions in $2^{n/3}$ oracle queries but requires random access to a quantum memory of about 2^{92} bits at these output sizes, and the best *known* implementable generic quantum collision attack remains the classical parallel one — the assessment ZIP-2005 adopts, flagged there as best-known rather than provably best.

One caveat attaches to Poseidon: its PRF security is a cryptanalytic heuristic with no standard-problem reduction, classically and quantumly alike (the *Crypto Guide*, §"Arithmetisation-friendly hashing: Poseidon"). A quantum adversary gains no identified structural advantage, and since nk stays off chain, recorded nullifiers remain unlinkable either way.

The survey's load-bearing consequence: the symmetric backbone survives a discrete-log break intact. A seed holder can still prove, by re-executing the hardened derivations, that their keys descend from their seed — ownership remains *provable* after the assumption that made it *enforceable* has fallen. This is the fact ZIP-2005's quantum recoverability builds on.

14.5 ZIP-2005: quantum recoverability

ZIP-2005 ("Ironwood Quantum Recoverability"; status Proposed, created 2025-03-31) is the deployed-track response to the prospective breaks. It does not make Orchard quantum-secure, and says so; what it arranges is that funds survive the eventual switch-off. Its activation is bound to the NU6.3 network upgrade (ZIP-258, Draft; consensus branch `0x37A5165B`, activation constants by pointer in §9.3), a scheduling that follows the NU6.2 emergency circuit correction (ZIP-257, Final) — which is why the originally planned earlier deployment slipped. The consensus half of the response — the seal and the turnstile — was constructed in §10; this subsection owns the note format and its security analysis.

The format half is the Ironwood note: note plaintexts carry lead byte `0x03` (§5.1), and with it the hashed trapdoor of Definition 4.3 — rcm derived from the `0x0B`-domain-separated $\mathsf{pre}_{\mathsf{rcm}}$ concatenating every note field, in place of the legacy `0x05` $\|\ \rho$ input. Every note field thereby traverses BLAKE2b on its way into the trapdoor. The purpose was promised in one sentence at the construction site (§4.2); here is the argument in full.

Under a CRQC, NoteCommit alone is not binding: its binding is Sinsemilla collision resistance, first on the integrity stack to fall (§14.3). The Ironwood derivation restores binding *conditionally*. Model PRFexpand as a random oracle. An adversary who would open one commitment to two distinct notes, both openings respecting the derivation, cannot vary any note field freely: every field enters $\mathsf{pre}_{\mathsf{rcm}}$, so any change re-randomises rcm, and landing two honestly-derived note–trapdoor pairs on the same commitment point is a collision-type event in the oracle — generically hard, with no group structure for Shor to exploit, in the sense Theorem 14.3 makes precise. The *composite* of the derivation and NoteCommit is therefore post-quantum binding — *provided a verifier checks the derivation*. Neither NoteCommit nor the deployed Action circuit changed, and the circuit checks only the commitment, never the derivation; the binding claim is conditional on that future check, which is exactly the Recovery Statement's job.

The key half is a rederivation path for $\mathsf{Commit}^{\mathsf{ivk}}$. The randomness $\mathsf{r}_{\mathsf{ivk}}$ descends from sk through PRFexpand (Construction 3.4) — or, where no single sk serves — for FROST-generated keys, threshold signing in which ask arises from distributed shares rather than from sk, and for hardware-held keys, where proof authority must remain separate from the on-device spend key — ZIP-2005 derives $\mathsf{r}_{\mathsf{ivk}}$ from a *quantum key* qk, produced by one BLAKE3 `derive_key` compression from a secret qsk, with signatures of knowledge — proofs of knowledge of a secret, bound to a message the way a signature is — over the transaction sighash tying the claimed keys to the spend. A caveat: the ZIP types that sighash only as a message hash, and pins no sighash algorithm.

The *Recovery Statement* (non-normative) is what a holder would one day prove, in a post-quantum

knowledge-sound proof system, over a re-hashed commitment tree: the key derivations — BindKeys, ZIP-2005's name for the derivations of ask, ak, and nk from sk, plus the $\mathsf{r}_{\mathsf{ivk}}$ branch above — then $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}$, $\mathsf{pk}_{\mathsf{d}} = [\mathsf{ivk}]\,\mathsf{g}_{\mathsf{d}}$, the ψ and rcm derivations, the note commitment, a membership path, the nullifier, and the α-re-randomisation tying the public rk to ak. The cost: 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, and a Merkle path, with the BLAKE2b compressions dominating.

The binding arguments carry published bounds, stated in the *quantum random-oracle model* (QROM): the random-oracle model of the classical proofs (the *Crypto Guide*, §"Security in the random oracle model and the forking lemma"), with the adversary permitted superposition queries to the oracle.

> **Theorem 14.3** (Ironwood binding bounds, as published)**.** *In the QROM, with q oracle queries and $N \approx 2^{254}$ the oracle output-space size (the Pallas scalar order, p_{Vesta} in this volume's notation),*
>
> $$\varepsilon_{\mathrm{kb}}^{\mathrm{QROM}} \le O(q^3/N), \qquad \varepsilon_{\mathrm{Spend}}^{\mathrm{QROM}} \le O(q^3/N) + \varepsilon_{\mathrm{kb}}^{\mathrm{QROM}},$$
>
> *for the key-binding game — that the derivations bind the recovered keys — and the Spendability game of the attack catalogue of §14.3, respectively (ZIP-2005).*

Idea. The quantum analogue of collision resistance is Unruh's *collapsing* property: measuring the adversary's superposition of hash inputs is undetectable to it once the outputs are measured. Fehr's theorem for HAIFA-mode hashes — the counter-carrying iteration mode BLAKE2b instantiates — reduces collapsing of the rcm derivation to modelling BLAKE2b's compression function as a random oracle, the same modelling the classical proofs already use. Both classical reductions — key binding and Spendability — are straight-line, running the adversary once without rewinding, and never reprogram the oracle, so they lift to the QROM unchanged; by Zhandry's compressed-oracle technique the classical $O(q^2/N)$ collision bound becomes $O(q^3/N)$. □

Evaluated, the bounds are comfortable. At $q \ll 2^{60}$ queries the worst-case bound is about 2^{-74}, and the practically achievable bound stays near the classical $q^2/N \approx 2^{-134}$: saturating the cubic bound needs the memory-infeasible BHT-style algorithm of §14.4.

The scope is deliberately narrow: binding only, Orchard only. ZIP-2005 repairs binding, not privacy — the HNDL exposure of §14.2 persists unchanged for every pool, Ironwood included. Sapling is excluded, with the rationale quoted: "to reduce overall protocol complexity and analysis effort" — the ZIP-32 Sapling derivations differ significantly from Orchard's hardened-only tree and would require their own analysis. The consequence is forced migration: Sapling, Sprout, and pre-Ironwood Orchard (lead byte 0x02) notes are unrecoverable, hence permanently unspendable once their protocols switch off — whence the wallet posture of §10.5: move all funds into Ironwood notes, and treat the sweep as ongoing, since non-recoverable notes may arrive at any time.

The bins of Remark 14.2, applied:

- *Specified.* The lead-byte-0x03 derivation, the qsk/qk key path, and the consensus rules sealing the Orchard pool (§10.1) are pinned at ZIP rigour — ZIP-2005 (Proposed), ZIP-229 and ZIP-258 (Draft), all at zips commit 69610984 — and are enforced from NU6.3 activation: every Ironwood output is a recoverable note.
- *Designed but unspecified.* The Recovery Protocol itself. ZIP-2005 presents the Recovery Statement in outline and states that its details are "subject to change"; a specification would still need to pin down the post-quantum proof system (a stated requirement is that none be assumed now), the collapsing hash for the re-hashed commitment tree, the linking commitments between circuits, and the concrete signature-of-knowledge instantiations.
- *Open problem.* Two items. A post-quantum proof system and signature scheme for *ongoing* spends — as opposed to one-shot recovery — exist nowhere as drafts (zcash/zips#1134 carries discussion

only). And the switch-off schedule: every recoverability guarantee is conditional on legacy switch-off preceding the first quantum forgery, and attacks landed in the exposure window — Spendability roadblocks that permanently block recovery, spend-authority theft, turnstile-bounded inflation (§14.3) — are not repaired retroactively.

14.6 Strategy: recoverability, not resistance

One more artefact completes the picture. Ragu, the recursive proof system under development for project Tachyon — a scalability programme beyond this volume's scope — is deliberately based on the elliptic-curve discrete-log problem over the same Pasta cycle, its authors citing "reduced concern (in the medium term)" for post-quantum soundness risks (ragu-tachyon book, `src/protocol/index.md`, commit `830bbcda`, 2026-07-05). Bin: designed but unspecified.

Read together, Ragu's choice and ZIP-2005 fix Zcash's quantum strategy precisely: *recoverability, not resistance*. Keep discrete-log cryptography while it stands, because its integrity failures are prospective and fork-mitigable (§14.3); pre-position notes so that funds provably survive the eventual transition (§14.5). The one cost this strategy cannot answer is the retroactive privacy channel: the HNDL corpus of §14.2 accrues damage now.

The strategy should also be read against the publicity. Public claims that Zcash will be "fully post-quantum" on a twelve-to-eighteen-month horizon, with ML-KEM and ML-DSA under test (CoinDesk, 2026-05-08), correspond to no ZIP, no draft, and no repository artefact at the pins of Remark 14.2; ZIP-2005 itself disclaims making the protocol secure against quantum adversaries. The record supports a narrower and better claim. On the day the assumptions fall, the recorded chain stays sealed to anyone without an address, the pools close before the first forgery is accepted — conditional on the schedule that remains an open problem — and every Ironwood note walks out of the wreckage provably owned. The value survives its owner's adversaries; that is where the journey of this volume ends.